

Программная реализация сетевого сервера

Создано системой Doxygen 1.9.4

1	Алфавитный указатель классов	1
1.1	Классы	1
2	Список файлов	3
2.1	Файлы	3
3	Классы	5
3.1	Класс Authenticator	5
3.1.1	Подробное описание	5
3.1.2	Методы	6
3.1.2.1	authenticate()	6
3.1.2.2	computeSHA1()	6
3.1.2.3	generateSalt()	7
3.2	Класс ClientBase	7
3.2.1	Подробное описание	8
3.2.2	Конструктор(ы)	8
3.2.2.1	ClientBase()	8
3.2.3	Методы	8
3.2.3.1	getPassword()	9
3.2.3.2	load()	9
3.2.3.3	userExists()	9
3.2.4	Данные класса	10
3.2.4.1	clients	10
3.2.4.2	filename	10
3.3	Структура ClientBaseFixture	10
3.3.1	Подробное описание	10
3.3.2	Конструктор(ы)	10
3.3.2.1	ClientBaseFixture()	11
3.3.2.2	~ClientBaseFixture()	11
3.3.3	Данные класса	11
3.3.3.1	testFilename	11
3.4	Класс ClientHandler	11
3.4.1	Подробное описание	12
3.4.2	Конструктор(ы)	13
3.4.2.1	ClientHandler()	13
3.4.3	Методы	13
3.4.3.1	authenticateClient()	13
3.4.3.2	handle()	14
3.4.3.3	receiveData()	14
3.4.3.4	receiveString()	15
3.4.3.5	receiveVectorsAndProcess()	15
3.4.3.6	sendData()	16
3.4.3.7	sendMessage()	17
3.4.3.8	sendSalt()	17

3.4.4 Данные класса	18
3.4.4.1 clientAddr	18
3.4.4.2 clientBase	18
3.4.4.3 clientSocket	18
3.4.4.4 logger	19
3.5 Класс CommandLineParser	19
3.5.1 Подробное описание	20
3.5.2 Конструктор(ы)	20
3.5.2.1 CommandLineParser()	20
3.5.3 Методы	20
3.5.3.1 getConfig()	20
3.5.3.2 getHelp()	21
3.5.3.3 parse()	21
3.5.4 Данные класса	21
3.5.4.1 config	21
3.5.4.2 helpText	22
3.6 Класс Logger	22
3.6.1 Подробное описание	23
3.6.2 Конструктор(ы)	23
3.6.2.1 Logger()	23
3.6.2.2 ~Logger()	24
3.6.3 Методы	24
3.6.3.1 error()	24
3.6.3.2 getCurrentTime()	24
3.6.3.3 info()	24
3.6.3.4 levelToString()	25
3.6.3.5 log()	25
3.6.3.6 warning()	25
3.6.4 Данные класса	27
3.6.4.1 logFile	27
3.6.4.2 logMutex	27
3.7 Структура LoggerFixture	27
3.7.1 Подробное описание	28
3.7.2 Конструктор(ы)	28
3.7.2.1 LoggerFixture()	28
3.7.2.2 ~LoggerFixture()	28
3.7.3 Методы	28
3.7.3.1 readLogFile()	28
3.7.4 Данные класса	28
3.7.4.1 testLogFile	28
3.8 Класс OutputRedirect	29
3.8.1 Подробное описание	29
3.8.2 Конструктор(ы)	29

3.8.2.1	OutputRedirect()	29
3.8.2.2	~OutputRedirect()	29
3.8.3	Методы	29
3.8.3.1	clear()	29
3.8.3.2	getOutput()	30
3.8.4	Данные класса	30
3.8.4.1	buffer	30
3.8.4.2	old_cerr	30
3.8.4.3	old_cout	30
3.9	Класс Server	30
3.9.1	Подробное описание	31
3.9.2	Конструктор(ы)	32
3.9.2.1	Server()	32
3.9.2.2	~Server()	32
3.9.3	Методы	32
3.9.3.1	cleanup()	32
3.9.3.2	initialize()	33
3.9.3.3	run()	33
3.9.3.4	stop()	33
3.9.4	Данные класса	33
3.9.4.1	clientBase	33
3.9.4.2	clientBaseFile	34
3.9.4.3	logFile	34
3.9.4.4	logger	34
3.9.4.5	port	34
3.9.4.6	running	34
3.9.4.7	serverSocket	35
3.10	Структура ServerConfig	35
3.10.1	Подробное описание	35
3.10.2	Данные класса	35
3.10.2.1	clientBaseFile	35
3.10.2.2	logFile	36
3.10.2.3	port	36
3.11	Класс VectorCalculator	36
3.11.1	Подробное описание	36
3.11.2	Методы	37
3.11.2.1	computeProduct()	37
4	Файлы	39
4.1	Файл Authenticator.cpp	39
4.2	Authenticator.cpp	39
4.3	Файл Authenticator.h	40
4.4	Authenticator.h	41

4.5 Файл ClientBase.cpp	41
4.6 ClientBase.cpp	42
4.7 Файл ClientBase.h	42
4.8 ClientBase.h	43
4.9 Файл ClientHandler.cpp	44
4.9.1 Макросы	44
4.9.1.1 be64toh	44
4.9.1.2 htobe64	44
4.10 ClientHandler.cpp	45
4.11 Файл ClientHandler.h	48
4.12 ClientHandler.h	49
4.13 Файл CommandLineParser.cpp	49
4.14 CommandLineParser.cpp	50
4.15 Файл CommandLineParser.h	51
4.16 CommandLineParser.h	52
4.17 Файл Logger.cpp	52
4.18 Logger.cpp	53
4.19 Файл Logger.h	54
4.19.1 Перечисления	55
4.19.1.1 LogLevel	55
4.20 Logger.h	55
4.21 Файл main.cpp	56
4.21.1 Функции	56
4.21.1.1 main()	56
4.21.1.2 signalHandler()	57
4.21.2 Переменные	57
4.21.2.1 server	57
4.22 main.cpp	58
4.23 Файл Server.cpp	58
4.24 Server.cpp	59
4.25 Файл Server.h	60
4.26 Server.h	61
4.27 Файл Test_Authenticator.cpp	61
4.27.1 Функции	62
4.27.1.1 SUITE()	62
4.28 Test_Authenticator.cpp	62
4.29 Файл Test_ClientBase.cpp	62
4.29.1 Функции	63
4.29.1.1 SUITE()	63
4.30 Test_ClientBase.cpp	63
4.31 Файл Test_CommandLineParser.cpp	64
4.31.1 Функции	65
4.31.1.1 SUITE()	65

4.32 Test_CommandLineParser.cpp	65
4.33 Файл Test_Logger.cpp	67
4.33.1 Функции	67
4.33.1.1 SUITE()	67
4.34 Test_Logger.cpp	68
4.35 Файл test_main.cpp	69
4.35.1 Функции	69
4.35.1.1 main()	69
4.36 test_main.cpp	69
4.37 Файл Test_ServerIntegration.cpp	70
4.37.1 Функции	70
4.37.1.1 SUITE()	70
4.38 Test_ServerIntegration.cpp	70
4.39 Файл Test_VectorCalculator.cpp	71
4.39.1 Функции	71
4.39.1.1 SUITE()	71
4.40 Test_VectorCalculator.cpp	71
4.41 Файл VectorCalculator.cpp	72
4.42 VectorCalculator.cpp	72
4.43 Файл VectorCalculator.h	73
4.44 VectorCalculator.h	74
Предметный указатель	75

Глава 1

Алфавитный указатель классов

1.1 Классы

Классы с их кратким описанием.

Authenticator	
Класс для аутентификации клиентов	5
ClientBase	
Класс для работы с базой данных клиентов	7
ClientBaseFixture	10
ClientHandler	
Класс для обработки клиентских подключений	11
CommandLineParser	
Класс для разбора командной строки	19
Logger	
Класс для логирования событий сервера	22
LoggerFixture	27
OutputRedirect	29
Server	
Основной класс сервера	30
ServerConfig	
Структура для хранения конфигурации сервера	35
VectorCalculator	
Класс для вычисления произведения элементов вектора	36

Глава 2

Список файлов

2.1 Файлы

Полный список файлов.

Authenticator.cpp	39
Authenticator.h	40
ClientBase.cpp	41
ClientBase.h	42
ClientHandler.cpp	44
ClientHandler.h	48
CommandLineParser.cpp	49
CommandLineParser.h	51
Logger.cpp	52
Logger.h	54
main.cpp	56
Server.cpp	58
Server.h	60
Test_Authenticator.cpp	61
Test_ClientBase.cpp	62
Test_CommandLineParser.cpp	64
Test_Logger.cpp	67
test_main.cpp	69
Test_ServerIntegration.cpp	70
Test_VectorCalculator.cpp	71
VectorCalculator.cpp	72
VectorCalculator.h	73

Глава 3

Классы

3.1 Класс Authenticator

Класс для аутентификации клиентов

```
#include <Authenticator.h>
```

Открытые статические члены

- static std::string [generateSalt](#) ()
Генерация случайной соли
- static std::string [computeSHA1](#) (const std::string &data)
Вычисление SHA1 хеша от данных
- static bool [authenticate](#) (const std::string &password, const std::string &salt, const std::string &clientHash)
Аутентификация клиента

3.1.1 Подробное описание

Класс для аутентификации клиентов

Автор

Гришин Николай

Версия

1.0

Дата

24.11.2025

Авторство

ИБСТ ПГУ

См. определение в файле [Authenticator.h](#) строка 19

3.1.2 Методы

3.1.2.1 authenticate()

```
bool Authenticator::authenticate (
    const std::string & password,
    const std::string & salt,
    const std::string & clientHash ) [static]
```

Аутентификация клиента

Аргументы

password	Пароль пользователя
salt	Соль для аутентификации
clientHash	Хеш от клиента

Возвращает

true если аутентификация успешна, false в противном случае

См. определение в файле [Authenticator.cpp](#) строка 51

3.1.2.2 computeSHA1()

```
std::string Authenticator::computeSHA1 (
    const std::string & data ) [static]
```

Вычисление SHA1 хеша от данных

Аргументы

data	Данные для хеширования
------	------------------------

Возвращает

Хеш в виде строки шестнадцатеричных цифр

См. определение в файле [Authenticator.cpp](#) строка 32

3.1.2.3 generateSalt()

```
std::string Authenticator::generateSalt ( ) [static]
```

Генерация случайной соли

Возвращает

Соль в виде строки из 16 шестнадцатеричных цифр

См. определение в файле [Authenticator.cpp](#) строка 13

Объявления и описания членов классов находятся в файлах:

- [Authenticator.h](#)
- [Authenticator.cpp](#)

3.2 Класс ClientBase

Класс для работы с базой данных клиентов

```
#include <ClientBase.h>
```

Открытые члены

- [ClientBase](#) (const std::string &filename)
Конструктор класса [ClientBase](#).
- bool [load](#) ()
Загрузка базы данных клиентов из файла
- bool [userExists](#) (const std::string &login) const
Проверка существования пользователя в базе
- std::string [getPassword](#) (const std::string &login) const
Получение пароля пользователя

Закрытые данные

- std::string [filename](#)
Имя файла с базой данных
- std::unordered_map< std::string, std::string > [clients](#)
Контейнер для хранения клиентов

3.2.1 Подробное описание

Класс для работы с базой данных клиентов

Автор

Гришин Николай

Версия

1.0

Дата

24.11.2025

Авторство

ИБСТ ПГУ

См. определение в файле [ClientBase.h](#) строка 14

3.2.2 Конструктор(ы)

3.2.2.1 ClientBase()

```
ClientBase::ClientBase (  
    const std::string & filename )
```

Конструктор класса [ClientBase](#).

Аргументы

filename	Имя файла с базой данных клиентов
----------	-----------------------------------

См. определение в файле [ClientBase.cpp](#) строка 10

3.2.3 Методы

3.2.3.1 getPassword()

```
std::string ClientBase::getPassword (
    const std::string & login ) const
```

Получение пароля пользователя

Аргументы

login	Логин пользователя
-------	--------------------

Возвращает

Пароль пользователя или пустую строку если пользователь не найден

См. определение в файле [ClientBase.cpp](#) строка 50

3.2.3.2 load()

```
bool ClientBase::load ( )
```

Загрузка базы данных клиентов из файла

Возвращает

true если загрузка успешна, false в противном случае

См. определение в файле [ClientBase.cpp](#) строка 16

3.2.3.3 userExists()

```
bool ClientBase::userExists (
    const std::string & login ) const
```

Проверка существования пользователя в базе

Аргументы

login	Логин пользователя
-------	--------------------

Возвращает

true если пользователь существует, false в противном случае

См. определение в файле [ClientBase.cpp](#) строка 41

3.2.4 Данные класса

3.2.4.1 clients

```
std::unordered_map<std::string, std::string> ClientBase::clients [private]
```

Контейнер для хранения клиентов

См. определение в файле [ClientBase.h](#) строка 44

3.2.4.2 filename

```
std::string ClientBase::filename [private]
```

Имя файла с базой данных

См. определение в файле [ClientBase.h](#) строка 43

Объявления и описания членов классов находятся в файлах:

- [ClientBase.h](#)
- [ClientBase.cpp](#)

3.3 Структура ClientBaseFixture

Открытые члены

- [ClientBaseFixture \(\)](#)
- [~ClientBaseFixture \(\)](#)

Открытые атрибуты

- `std::string testFilename = "test_clients.txt"`

3.3.1 Подробное описание

См. определение в файле [Test_ClientBase.cpp](#) строка 5

3.3.2 Конструктор(ы)

3.3.2.1 ClientBaseFixture()

`ClientBaseFixture::ClientBaseFixture ( ) [inline]`

См. определение в файле [Test_ClientBase.cpp](#) строка 8

3.3.2.2 ~ClientBaseFixture()

`ClientBaseFixture::~~ClientBaseFixture ( ) [inline]`

См. определение в файле [Test_ClientBase.cpp](#) строка 17

3.3.3 Данные класса

3.3.3.1 testFilename

```
std::string ClientBaseFixture::testFilename = "test_clients.txt"
```

См. определение в файле [Test_ClientBase.cpp](#) строка 6

Объявления и описания членов структуры находятся в файле:

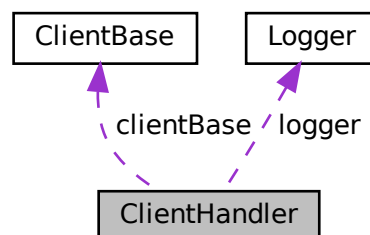
- [Test_ClientBase.cpp](#)

3.4 Класс ClientHandler

Класс для обработки клиентских подключений

```
#include <ClientHandler.h>
```

Граф связей класса ClientHandler:



Открытые члены

- [ClientHandler](#) (int [clientSocket](#), const sockaddr_in &[clientAddr](#), [ClientBase](#) &[clientBase](#), [Logger](#) &[logger](#))
Конструктор класса [ClientHandler](#).
- void [handle](#) ()
Основной метод обработки клиента

Закрытые члены

- bool [authenticateClient](#) ()
Аутентификация клиента
- bool [receiveVectorsAndProcess](#) ()
Прием и обработка векторов от клиента
- bool [sendMessage](#) (const std::string &message)
Отправка текстового сообщения клиенту
- bool [receiveData](#) (void *buffer, size_t size)
Прием данных от клиента
- bool [sendData](#) (const void *data, size_t size)
Отправка данных клиенту
- bool [receiveString](#) (std::string &result, size_t maxSize=256)
Прием строки от клиента
- bool [sendSalt](#) (const std::string &salt)
Отправка соли клиенту

Закрытые данные

- int [clientSocket](#)
Сокет клиента
- sockaddr_in [clientAddr](#)
Адрес клиента
- [ClientBase](#) & [clientBase](#)
Ссылка на базу клиентов
- [Logger](#) & [logger](#)
Ссылка на логгер

3.4.1 Подробное описание

Класс для обработки клиентских подключений

Автор

Гришин Николай

Версия

1.0

Дата

24.11.2025

Авторство

ИБСТ ПГУ

См. определение в файле [ClientHandler.h](#) строка 21

3.4.2 Конструктор(ы)

3.4.2.1 ClientHandler()

```
ClientHandler::ClientHandler (
    int clientSocket,
    const sockaddr_in & clientAddr,
    ClientBase & clientBase,
    Logger & logger )
```

Конструктор класса [ClientHandler](#).

Аргументы

clientSocket	Сокет клиента
clientAddr	Адрес клиента
clientBase	База данных клиентов
logger	Логгер для записи событий

См. определение в файле [ClientHandler.cpp](#) строка 25

3.4.3 Методы

3.4.3.1 authenticateClient()

```
bool ClientHandler::authenticateClient ( ) [private]
```

Аутентификация клиента

Аутентификация клиента по протоколу

Возвращает

true если аутентификация успешна, false в противном случае
true если аутентификация успешна, false в противном случае

Выполняет следующие шаги:

1. Прием логина от клиента
2. Проверка существования пользователя в базе
3. Генерация и отправка соли
4. Прием хеша от клиента
5. Проверка хеша аутентификации

См. определение в файле [ClientHandler.cpp](#) строка 71

3.4.3.2 handle()

```
void ClientHandler::handle ( )
```

Основной метод обработки клиента

Основной метод обработки клиентского подключения

Выполняет аутентификацию клиента и обработку векторов данных

См. определение в файле [ClientHandler.cpp](#) строка 34

3.4.3.3 receiveData()

```
bool ClientHandler::receiveData (
    void * buffer,
    size_t size ) [private]
```

Прием данных от клиента

Прием бинарных данных от клиента

Аргументы

buffer	Буфер для приема данных
size	Размер данных

Возвращает

true если прием успешен, false в противном случае

Аргументы

buffer	Буфер для приема данных
size	Размер данных для приема

Возвращает

true если прием успешен, false в противном случае

Читает точное количество байт без поиска терминатора

См. определение в файле [ClientHandler.cpp](#) строка 231

3.4.3.4 receiveString()

```
bool ClientHandler::receiveString (
    std::string & result,
    size_t maxSize = 256 ) [private]
```

Прием строки от клиента

Аргументы

result	Строка для результата
maxSize	Максимальный размер строки

Возвращает

true если прием успешен, false в противном случае

Аргументы

result	Строка для результата
maxSize	Максимальный размер строки

Возвращает

true если прием успешен, false в противном случае

Использует различные стратегии для приема строки:

- Peek для анализа буфера
- Поиск терминаторов (`\0`, `\r`)
- Резервное чтение побайтно

См. определение в файле [ClientHandler.cpp](#) строка 283

3.4.3.5 receiveVectorsAndProcess()

```
bool ClientHandler::receiveVectorsAndProcess ( ) [private]
```

Прием и обработка векторов от клиента

Прием и обработка векторов данных от клиента

Возвращает

true если обработка успешна, false в противном случае
true если обработка успешна, false в противном случае

Выполняет следующие шаги:

1. Прием количества векторов
2. Для каждого вектора:
 - Прием размера вектора
 - Прием данных вектора
 - Вычисление произведения элементов
 - Отправка результата клиенту

См. определение в файле [ClientHandler.cpp](#) строка 144

3.4.3.6 sendData()

```
bool ClientHandler::sendData (  
    const void * data,  
    size_t size ) [private]
```

Отправка данных клиенту

Отправка бинарных данных клиенту

Аргументы

data	Указатель на данные
size	Размер данных

Возвращает

true если отправка успешна, false в противном случае

Аргументы

data	Указатель на данные для отправки
size	Размер данных для отправки

Возвращает

true если отправка успешна, false в противном случае

Отправляет точное количество байт

См. определение в файле [ClientHandler.cpp](#) строка 257

3.4.3.7 sendMessage()

```
bool ClientHandler::sendMessage (
    const std::string & message ) [private]
```

Отправка текстового сообщения клиенту

Аргументы

message	Сообщение для отправки
---------	------------------------

Возвращает

true если отправка успешна, false в противном случае

Аргументы

message	Сообщение для отправки
---------	------------------------

Возвращает

true если отправка успешна, false в противном случае

Заметки

Отправляет сообщение с нулевым терминатором

См. определение в файле [ClientHandler.cpp](#) строка 219

3.4.3.8 sendSalt()

```
bool ClientHandler::sendSalt (
    const std::string & salt ) [private]
```

Отправка соли клиенту

Аргументы

salt	Соль для отправки
------	-------------------

Возвращает

true если отправка успешна, false в противном случае

Аргументы

salt	Соль для отправки (16 байт)
------	-----------------------------

Возвращает

true если отправка успешна, false в противном случае

Заметки

Отправляет ровно 16 байт без нулевого терминатора

См. определение в файле [ClientHandler.cpp](#) строка 208

3.4.4 Данные класса

3.4.4.1 clientAddr

sockaddr_in ClientHandler::clientAddr [private]

Адрес клиента

См. определение в файле [ClientHandler.h](#) строка 40

3.4.4.2 clientBase

[ClientBase&](#) ClientHandler::clientBase [private]

Ссылка на базу клиентов

См. определение в файле [ClientHandler.h](#) строка 41

3.4.4.3 clientSocket

int ClientHandler::clientSocket [private]

Сокет клиента

См. определение в файле [ClientHandler.h](#) строка 39

3.4.4.4 logger

`Logger& ClientHandler::logger` [private]

Ссылка на логгер

См. определение в файле [ClientHandler.h](#) строка 42

Объявления и описания членов классов находятся в файлах:

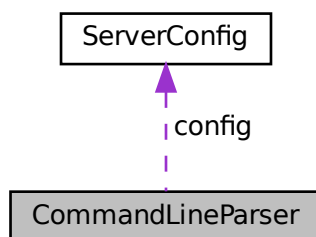
- [ClientHandler.h](#)
- [ClientHandler.cpp](#)

3.5 Класс CommandLineParser

Класс для разбора командной строки

```
#include <CommandLineParser.h>
```

Граф связей класса CommandLineParser:



Открытые члены

- [CommandLineParser](#) ()
Конструктор класса [CommandLineParser](#).
- `bool` [parse](#) (int argc, char *argv[])
Разбор аргументов командной строки
- [ServerConfig](#) [getConfig](#) () const
Получение конфигурации сервера
- `std::string` [getHelp](#) () const
Получение текста справки

Закрытые данные

- [ServerConfig](#) [config](#)
Конфигурация сервера
- `std::string` [helpText](#)
Текст справки

3.5.1 Подробное описание

Класс для разбора командной строки

Автор

Гришин Николай

Версия

1.0

Дата

24.11.2025

Авторство

ИБСТ ПГУ

См. определение в файле [CommandLineParser.h](#) строка 22

3.5.2 Конструктор(ы)

3.5.2.1 CommandLineParser()

`CommandLineParser::CommandLineParser ( )`

Конструктор класса [CommandLineParser](#).

См. определение в файле [CommandLineParser.cpp](#) строка 10

3.5.3 Методы

3.5.3.1 getConfig()

[ServerConfig](#) `CommandLineParser::getConfig ( ) const` [inline]

Получение конфигурации сервера

Возвращает

Структура [ServerConfig](#) с настройками

См. определение в файле [CommandLineParser.h](#) строка 41

3.5.3.2 getHelp()

```
std::string CommandLineParser::getHelp ( ) const
```

Получение текста справки

Возвращает

Строка с текстом справки

См. определение в файле [CommandLineParser.cpp](#) строка 100

3.5.3.3 parse()

```
bool CommandLineParser::parse (
    int argc,
    char * argv[] )
```

Разбор аргументов командной строки

Аргументы

argc	Количество аргументов
argv	Массив аргументов

Возвращает

true если разбор успешен, false в противном случае

См. определение в файле [CommandLineParser.cpp](#) строка 31

3.5.4 Данные класса

3.5.4.1 config

```
ServerConfig CommandLineParser::config [private]
```

Конфигурация сервера

См. определение в файле [CommandLineParser.h](#) строка 50

3.5.4.2 helpText

```
std::string CommandLineParser::helpText [private]
```

Текст справки

См. определение в файле [CommandLineParser.h](#) строка 51

Объявления и описания членов классов находятся в файлах:

- [CommandLineParser.h](#)
- [CommandLineParser.cpp](#)

3.6 Класс Logger

Класс для логирования событий сервера

```
#include <Logger.h>
```

Открытые члены

- [Logger](#) (const std::string &filename)
Конструктор класса [Logger](#).
- [~Logger](#) ()
Деструктор класса [Logger](#).
- void [log](#) (LogLevel level, const std::string &message)
Основной метод логирования
- void [info](#) (const std::string &message)
Логирование информационного сообщения
- void [warning](#) (const std::string &message)
Логирование предупреждения
- void [error](#) (const std::string &message)
Логирование ошибки

Закрытые члены

- std::string [getCurrentTime](#) ()
Получение текущего времени в формате строки
- std::string [levelToString](#) (LogLevel level)
Преобразование уровня логирования в строку

Закрытые данные

- std::ofstream [logFile](#)
Файл для записи логов
- std::mutex [logMutex](#)
Мьютекс для синхронизации доступа к файлу

3.6.1 Подробное описание

Класс для логирования событий сервера

Автор

Гришин Николай

Версия

1.0

Дата

24.11.2025

Авторство

ИБСТ ПГУ

См. определение в файле [Logger.h](#) строка 28

3.6.2 Конструктор(ы)

3.6.2.1 Logger()

```
Logger::Logger (  
    const std::string & filename )
```

Конструктор класса [Logger](#).

Аргументы

filename	Имя файла для логирования
----------	---------------------------

Исключения

std::runtime_error	если файл не может быть открыт
--------------------	--------------------------------

См. определение в файле [Logger.cpp](#) строка 10

3.6.2.2 ~Logger()

```
Logger::~Logger ( )
```

Деструктор класса [Logger](#).

См. определение в файле [Logger.cpp](#) строка 20

3.6.3 Методы

3.6.3.1 error()

```
void Logger::error (
    const std::string & message ) [inline]
```

Логирование ошибки

Аргументы

message	Сообщение для записи
---------	----------------------

См. определение в файле [Logger.h](#) строка 65

3.6.3.2 getCurrentTime()

```
std::string Logger::getCurrentTime ( ) [private]
```

Получение текущего времени в формате строки

Возвращает

Строка с текущим временем

См. определение в файле [Logger.cpp](#) строка 47

3.6.3.3 info()

```
void Logger::info (
    const std::string & message ) [inline]
```

Логирование информационного сообщения

Аргументы

message	Сообщение для записи
---------	----------------------

См. определение в файле [Logger.h](#) строка 53

3.6.3.4 levelToString()

```
std::string Logger::levelToString (
    LogLevel level ) [private]
```

Преобразование уровня логирования в строку

Аргументы

level	Уровень логирования
-------	---------------------

Возвращает

Строковое представление уровня

См. определение в файле [Logger.cpp](#) строка 71

3.6.3.5 log()

```
void Logger::log (
    LogLevel level,
    const std::string & message )
```

Основной метод логирования

Аргументы

level	Уровень логирования
message	Сообщение для записи

См. определение в файле [Logger.cpp](#) строка 31

3.6.3.6 warning()

```
void Logger::warning (
    const std::string & message ) [inline]
```

Логирование предупреждения

Аргументы

message	Сообщение для записи
---------	----------------------

См. определение в файле [Logger.h](#) строка [59](#)

3.6.4 Данные класса

3.6.4.1 logFile

`std::ofstream Logger::logFile` [private]

Файл для записи логов

См. определение в файле [Logger.h](#) строка [68](#)

3.6.4.2 logMutex

`std::mutex Logger::logMutex` [private]

Мьютекс для синхронизации доступа к файлу

См. определение в файле [Logger.h](#) строка [69](#)

Объявления и описания членов классов находятся в файлах:

- [Logger.h](#)
- [Logger.cpp](#)

3.7 Структура LoggerFixture

Открытые члены

- [LoggerFixture](#) ()
- [~LoggerFixture](#) ()
- `std::string` [readLogFile](#) ()

Открытые атрибуты

- `std::string` [testLogFile](#) = "test_log.txt"

3.7.1 Подробное описание

См. определение в файле [Test_Logger.cpp](#) строка 6

3.7.2 Конструктор(ы)

3.7.2.1 LoggerFixture()

```
LoggerFixture::LoggerFixture ( ) [inline]
```

См. определение в файле [Test_Logger.cpp](#) строка 9

3.7.2.2 ~LoggerFixture()

```
LoggerFixture::~LoggerFixture ( ) [inline]
```

См. определение в файле [Test_Logger.cpp](#) строка 13

3.7.3 Методы

3.7.3.1 readLogFile()

```
std::string LoggerFixture::readLogFile ( ) [inline]
```

См. определение в файле [Test_Logger.cpp](#) строка 17

3.7.4 Данные класса

3.7.4.1 testLogFile

```
std::string LoggerFixture::testLogFile = "test_log.txt"
```

См. определение в файле [Test_Logger.cpp](#) строка 7

Объявления и описания членов структуры находятся в файле:

- [Test_Logger.cpp](#)

3.8 Класс OutputRedirect

Открытые члены

- [OutputRedirect](#) ()
- [~OutputRedirect](#) ()
- `std::string` [getOutput](#) () const
- `void` [clear](#) ()

Закрытые данные

- `std::stringstream` [buffer](#)
- `std::streambuf *` [old_cout](#)
- `std::streambuf *` [old_cerr](#)

3.8.1 Подробное описание

См. определение в файле [Test_CommandLineParser.cpp](#) строка 7

3.8.2 Конструктор(ы)

3.8.2.1 OutputRedirect()

`OutputRedirect::OutputRedirect ( )` [inline]

См. определение в файле [Test_CommandLineParser.cpp](#) строка 13

3.8.2.2 ~OutputRedirect()

`OutputRedirect::~OutputRedirect ( )` [inline]

См. определение в файле [Test_CommandLineParser.cpp](#) строка 18

3.8.3 Методы

3.8.3.1 clear()

`void OutputRedirect::clear ( )` [inline]

См. определение в файле [Test_CommandLineParser.cpp](#) строка 27

3.8.3.2 `getOutput()`

```
std::string OutputRedirect::getOutput ( ) const [inline]
```

См. определение в файле [Test_CommandLineParser.cpp](#) строка 23

3.8.4 Данные класса

3.8.4.1 `buffer`

```
std::stringstream OutputRedirect::buffer [private]
```

См. определение в файле [Test_CommandLineParser.cpp](#) строка 8

3.8.4.2 `old_cerr`

```
std::streambuf* OutputRedirect::old_cerr [private]
```

См. определение в файле [Test_CommandLineParser.cpp](#) строка 10

3.8.4.3 `old_cout`

```
std::streambuf* OutputRedirect::old_cout [private]
```

См. определение в файле [Test_CommandLineParser.cpp](#) строка 9

Объявления и описания членов класса находятся в файле:

- [Test_CommandLineParser.cpp](#)

3.9 Класс Server

Основной класс сервера

```
#include <Server.h>
```

Открытые члены

- `Server` (`const std::string &clientBaseFile`, `const std::string &logFile`, `int port`)
Конструктор класса `Server`.
- `~Server` ()
Деструктор класса `Server`.
- `bool initialize` ()
Инициализация сервера
- `void run` ()
Запуск основного цикла сервера
- `void stop` ()
Остановка сервера

Закрытые члены

- `void cleanup` ()
Очистка ресурсов сервера

Закрытые данные

- `std::string clientBaseFile`
Имя файла базы клиентов
- `std::string logFile`
Имя файла журнала
- `int port`
Порт сервера
- `int serverSocket`
Сокет сервера
- `bool running`
Флаг работы сервера
- `std::unique_ptr< ClientBase > clientBase`
Указатель на базу клиентов
- `std::unique_ptr< Logger > logger`
Указатель на логгер

3.9.1 Подробное описание

Основной класс сервера

Автор

Гришин Николай

Версия

1.0

Дата

24.11.2025

Авторство

ИБСТ ПГУ

См. определение в файле `Server.h` строка 21

3.9.2 Конструктор(ы)

3.9.2.1 Server()

```
Server::Server (
    const std::string & clientBaseFile,
    const std::string & logFile,
    int port )
```

Конструктор класса [Server](#).

Аргументы

clientBaseFile	Файл базы клиентов
logFile	Файл журнала
port	Порт для прослушивания

См. определение в файле [Server.cpp](#) строка [12](#)

3.9.2.2 ~Server()

```
Server::~Server ( )
```

Деструктор класса [Server](#).

См. определение в файле [Server.cpp](#) строка [19](#)

3.9.3 Методы

3.9.3.1 cleanup()

```
void Server::cleanup ( ) [private]
```

Очистка ресурсов сервера

См. определение в файле [Server.cpp](#) строка [120](#)

3.9.3.2 initialize()

```
bool Server::initialize ( )
```

Инициализация сервера

Возвращает

true если инициализация успешна, false в противном случае

См. определение в файле [Server.cpp](#) строка 27

3.9.3.3 run()

```
void Server::run ( )
```

Запуск основного цикла сервера

См. определение в файле [Server.cpp](#) строка 84

3.9.3.4 stop()

```
void Server::stop ( )
```

Остановка сервера

См. определение в файле [Server.cpp](#) строка 108

3.9.4 Данные класса

3.9.4.1 clientBase

```
std::unique_ptr<ClientBase> Server::clientBase [private]
```

Указатель на базу клиентов

См. определение в файле [Server.h](#) строка 59

3.9.4.2 clientBaseFile

```
std::string Server::clientBaseFile [private]
```

Имя файла базы клиентов

См. определение в файле [Server.h](#) строка 53

3.9.4.3 logFile

```
std::string Server::logFile [private]
```

Имя файла журнала

См. определение в файле [Server.h](#) строка 54

3.9.4.4 logger

```
std::unique_ptr<Logger> Server::logger [private]
```

Указатель на логгер

См. определение в файле [Server.h](#) строка 60

3.9.4.5 port

```
int Server::port [private]
```

Порт сервера

См. определение в файле [Server.h](#) строка 55

3.9.4.6 running

```
bool Server::running [private]
```

Флаг работы сервера

См. определение в файле [Server.h](#) строка 57

3.9.4.7 serverSocket

```
int Server::serverSocket [private]
```

Сокет сервера

См. определение в файле [Server.h](#) строка 56

Объявления и описания членов классов находятся в файлах:

- [Server.h](#)
- [Server.cpp](#)

3.10 Структура ServerConfig

Структура для хранения конфигурации сервера

```
#include <CommandLineParser.h>
```

Открытые атрибуты

- `std::string` [clientBaseFile](#)
Файл базы клиентов
- `std::string` [logFile](#)
Файл журнала
- `int` [port](#)
Порт сервера

3.10.1 Подробное описание

Структура для хранения конфигурации сервера

См. определение в файле [CommandLineParser.h](#) строка 9

3.10.2 Данные класса

3.10.2.1 clientBaseFile

```
std::string ServerConfig::clientBaseFile
```

Файл базы клиентов

См. определение в файле [CommandLineParser.h](#) строка 10

3.10.2.2 logFile

`std::string ServerConfig::logFile`

Файл журнала

См. определение в файле [CommandLineParser.h](#) строка 11

3.10.2.3 port

`int ServerConfig::port`

Порт сервера

См. определение в файле [CommandLineParser.h](#) строка 12

Объявления и описания членов структуры находятся в файле:

- [CommandLineParser.h](#)

3.11 Класс VectorCalculator

Класс для вычисления произведения элементов вектора

`#include <VectorCalculator.h>`

Открытые статические члены

- `static uint64_t computeProduct (const std::vector< uint64_t > &vector)`
Вычисление произведения элементов вектора

3.11.1 Подробное описание

Класс для вычисления произведения элементов вектора

Автор

Гришин Николай

Версия

1.0

Дата

24.11.2025

Авторство

ИБСТ ПГУ

См. определение в файле [VectorCalculator.h](#) строка 14

3.11.2 Методы

3.11.2.1 computeProduct()

```
uint64_t VectorCalculator::computeProduct (
    const std::vector< uint64_t > & vector ) [static]
```

Вычисление произведения элементов вектора

Аргументы

vector	Вектор значений для вычисления
--------	--------------------------------

Возвращает

Произведение элементов вектора

Заметки

При переполнении возвращается UINT64_MAX

См. определение в файле [VectorCalculator.cpp](#) строка 12

Объявления и описания членов классов находятся в файлах:

- [VectorCalculator.h](#)
- [VectorCalculator.cpp](#)

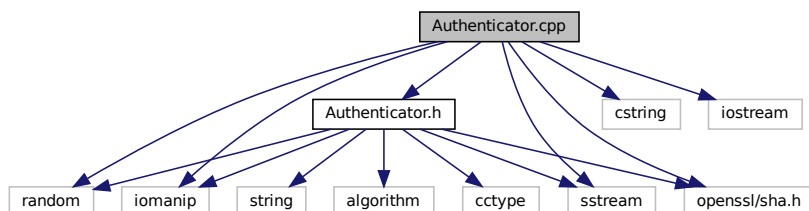
Глава 4

Файлы

4.1 Файл Authenticator.cpp

```
#include "Authenticator.h"  
#include <random>  
#include <iomanip>  
#include <sstream>  
#include <openssl/sha.h>  
#include <cstring>  
#include <iostream>
```

Граф включаемых заголовочных файлов для Authenticator.cpp:



4.2 Authenticator.cpp

См. документацию.

```
00001 #include "Authenticator.h"  
00002 #include <random>  
00003 #include <iomanip>  
00004 #include <sstream>  
00005 #include <openssl/sha.h>  
00006 #include <cstring>  
00007 #include <iostream>  
00008  
00013 std::string Authenticator::generateSalt() {  
00014     std::random_device rd;  
00015     std::mt19937_64 gen(rd());  
00016     std::uniform_int_distribution<uint64_t> dis;  
00017  
00018     uint64_t salt = dis(gen);  
00019  
00020     std::stringstream ss;
```

```

00021     ss << std::hex << std::setw(16) << std::setfill('0') << salt;
00022     std::string result = ss.str();
00023
00024     return result;
00025 }
00026
00032 std::string Authenticator::computeSHA1(const std::string& data) {
00033     unsigned char hash[SHA_DIGEST_LENGTH];
00034     SHA1(reinterpret_cast<const unsigned char*>(data.c_str()), data.length(), hash);
00035
00036     std::stringstream ss;
00037     for (int i = 0; i < SHA_DIGEST_LENGTH; i++) {
00038         ss << std::hex << std::setw(2) << std::setfill('0') << static_cast<unsigned int>(hash[i]);
00039     }
00040
00041     return ss.str();
00042 }
00043
00051 bool Authenticator::authenticate(const std::string& password, const std::string& salt, const std::string& clientHash) {
00052     std::string combined = salt + password;
00053     std::string serverHash = computeSHA1(combined);
00054
00055     bool match = true;
00056     for (size_t i = 0; i < serverHash.length(); i++) {
00057         char serverChar = std::tolower(serverHash[i]);
00058         char clientChar = std::tolower(clientHash[i]);
00059
00060         if (serverChar != clientChar) {
00061             match = false;
00062             break;
00063         }
00064     }
00065
00066     return match;
00067 }

```

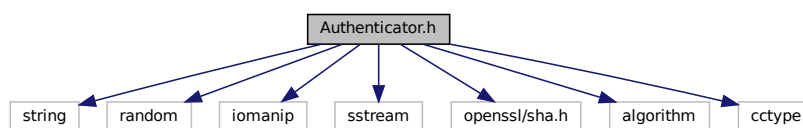
4.3 Файл Authenticator.h

```

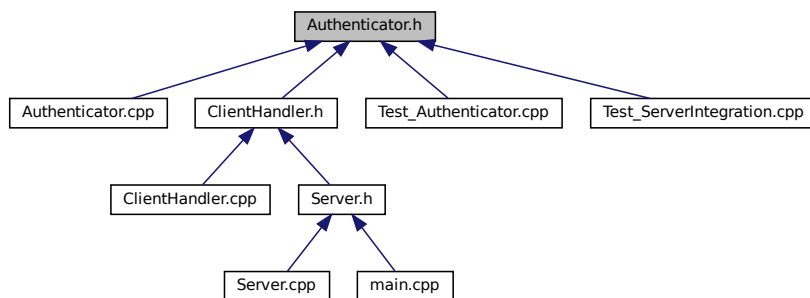
#include <string>
#include <random>
#include <iomanip>
#include <sstream>
#include <openssl/sha.h>
#include <algorithm>
#include <cctype>

```

Граф включаемых заголовочных файлов для Authenticator.h:



Граф файлов, в которые включается этот файл:



Классы

- class [Authenticator](#)
Класс для аутентификации клиентов

4.4 Authenticator.h

[См. документацию.](#)

```

00001 #ifndef AUTHENTICATOR_H
00002 #define AUTHENTICATOR_H
00003
00004 #include <string>
00005 #include <random>
00006 #include <iomanip>
00007 #include <sstream>
00008 #include <openssl/sha.h>
00009 #include <algorithm>
00010 #include <cctype>
00011
00019 class Authenticator {
00020 public:
00025     static std::string generateSalt();
00026
00032     static std::string computeSHA1(const std::string& data);
00033
00041     static bool authenticate(const std::string& password, const std::string& salt, const std::string& clientHash);
00042 };
00043
00044 #endif
  
```

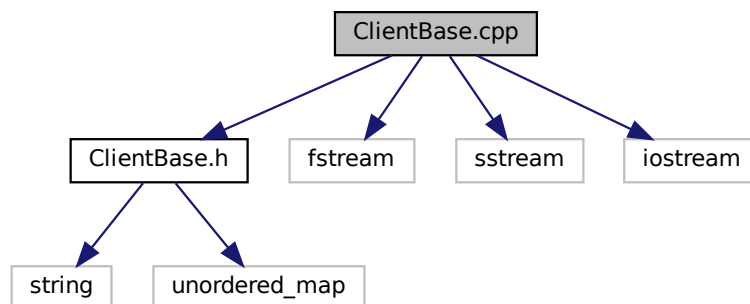
4.5 Файл ClientBase.cpp

```

#include "ClientBase.h"
#include <fstream>
#include <sstream>
  
```

```
#include <iostream>
```

Граф включаемых заголовочных файлов для ClientBase.cpp:



4.6 ClientBase.cpp

[См. документацию.](#)

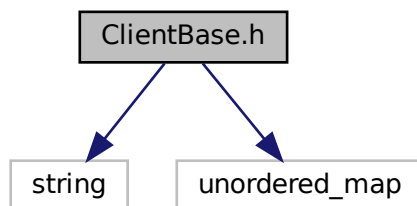
```

00001 #include "ClientBase.h"
00002 #include <fstream>
00003 #include <sstream>
00004 #include <iostream>
00005
00010 ClientBase::ClientBase(const std::string& filename) : filename(filename) {}
00011
00016 bool ClientBase::load() {
00017     std::ifstream file(filename);
00018     if (!file.is_open()) {
00019         return false;
00020     }
00021
00022     std::string line;
00023     while (std::getline(file, line)) {
00024         size_t pos = line.find(':');
00025         if (pos != std::string::npos) {
00026             std::string login = line.substr(0, pos);
00027             std::string password = line.substr(pos + 1);
00028             clients[login] = password;
00029         }
00030     }
00031
00032     file.close();
00033     return true;
00034 }
00035
00041 bool ClientBase::userExists(const std::string& login) const {
00042     return clients.find(login) != clients.end();
00043 }
00044
00050 std::string ClientBase::getPassword(const std::string& login) const {
00051     auto it = clients.find(login);
00052     return (it != clients.end()) ? it->second : "";
00053 }
  
```

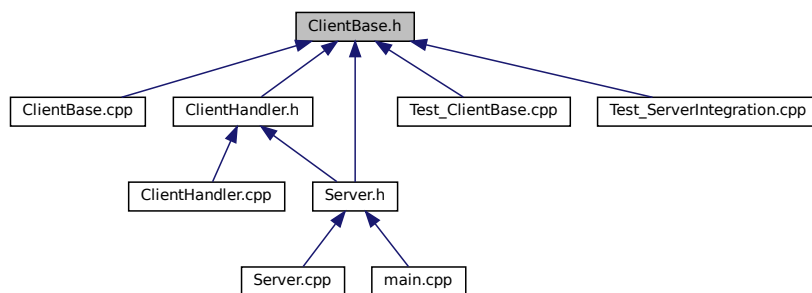
4.7 Файл ClientBase.h

```
#include <string>
#include <unordered_map>
```

Граф включаемых заголовочных файлов для ClientBase.h:



Граф файлов, в которые включается этот файл:



Классы

- class [ClientBase](#)

Класс для работы с базой данных клиентов

4.8 ClientBase.h

[См. документацию.](#)

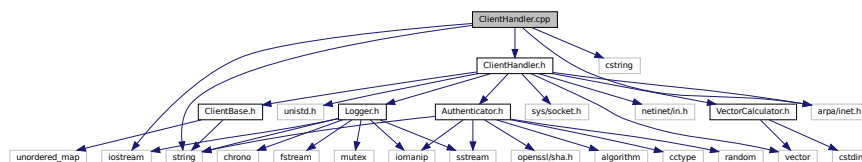
```

00001 #ifndef CLIENTBASE_H
00002 #define CLIENTBASE_H
00003
00004 #include <string>
00005 #include <unordered_map>
00006
00014 class ClientBase {
00015 public:
00020   ClientBase(const std::string& filename);
00021
00026   bool load();
00027
00033   bool userExists(const std::string& login) const;
00034
00040   std::string getPassword(const std::string& login) const;
00041
00042 private:
00043   std::string filename;
00044   std::unordered_map<std::string, std::string> clients;
00045 };
00046
00047 #endif
  
```

4.9 Файл ClientHandler.cpp

```
#include "ClientHandler.h"
#include <iostream>
#include <string>
#include <cstring>
#include <arpa/inet.h>
```

Граф включаемых заголовочных файлов для ClientHandler.cpp:



Макросы

- `#define be64toh(x) __bswap_64(x)`
- `#define htobe64(x) __bswap_64(x)`

4.9.1 Макросы

4.9.1.1 be64toh

```
#define be64toh(
    x ) __bswap_64(x)
```

См. определение в файле [ClientHandler.cpp](#) строка 10

4.9.1.2 htobe64

```
#define htobe64(
    x ) __bswap_64(x)
```

См. определение в файле [ClientHandler.cpp](#) строка 11

4.10 ClientHandler.cpp

См. документацию.

```

00001 #include "ClientHandler.h"
00002 #include <iostream>
00003 #include <string>
00004 #include <cstring>
00005 #include <arpa/inet.h> // для be64toh/htobe64
00006
00007 #ifndef be64toh
00008 // Если макросы не определены, создаем свои
00009 #if __BYTE_ORDER == __LITTLE_ENDIAN
00010 #define be64toh(x) __bswap_64(x)
00011 #define hto64(x) __bswap_64(x)
00012 #else
00013 #define be64toh(x) (x)
00014 #define hto64(x) (x)
00015 #endif
00016 #endif
00017
00025 ClientHandler::ClientHandler(int clientSocket, const sockaddr_in& clientAddr,
00026                               ClientBase& clientBase, Logger& logger)
00027 : clientSocket(clientSocket), clientAddr(clientAddr),
00028   clientBase(clientBase), logger(logger) {}
00029
00034 void ClientHandler::handle() {
00035     std::string clientIP = inet_ntoa(clientAddr.sin_addr);
00036     int clientPort = ntohs(clientAddr.sin_port);
00037
00038     logger.info("Client connected from " + clientIP + ":" + std::to_string(clientPort));
00039
00040     try {
00041         if (!authenticateClient()) {
00042             logger.warning("Authentication failed for client " + clientIP);
00043             close(clientSocket);
00044             return;
00045         }
00046
00047         logger.info("Client " + clientIP + " authenticated successfully");
00048
00049         if (!receiveVectorsAndProcess()) {
00050             logger.warning("Vector processing failed for client " + clientIP);
00051         }
00052     } catch (const std::exception& e) {
00053         logger.error("Error handling client " + clientIP + ": " + e.what());
00054     }
00055
00056     close(clientSocket);
00057     logger.info("Client " + clientIP + " disconnected");
00058 }
00059
00060
00071 bool ClientHandler::authenticateClient() {
00072
00073     // Шаг 1: Получаем логин
00074     std::string login;
00075     if (!receiveString(login)) {
00076         logger.error("Failed to receive login");
00077         return false;
00078     }
00079
00080     // Удаляем все пробелы из имени пользователя
00081     login.erase(0, login.find_first_not_of(" \t\n\r"));
00082     login.erase(login.find_last_not_of(" \t\n\r") + 1);
00083
00084     logger.info("Received login: " + login + "");
00085
00086     // Шаг 2: Проверяем, существует ли пользователь - ВОЗВРАЩАЕМ ПРОВЕРКУ
00087     if (!clientBase.userExists(login)) {
00088         logger.warning("User not found: " + login);
00089         sendMessage("ERR");
00090         return false;
00091     }
00092
00093     // ИСПОЛЬЗУЕМ ПАРОЛЬ ИЗ БАЗЫ ДАННЫХ
00094     std::string password = clientBase.getPassword(login);
00095
00096     // Шаг 3: Генерируем и отправляем соль
00097     std::string salt = Authenticator::generateSalt();
00098     logger.info("Generated salt for user " + login + ": " + salt);
00099
00100     if (!sendSalt(salt)) {
00101         logger.error("Failed to send salt");
00102         return false;
00103     }

```

```

00104
00105 // Шаг 4: Получаем хэш
00106 std::string clientHash;
00107
00108 // Используем receiveString для хеша тоже
00109 if (!receiveString(clientHash)) {
00110     logger.error("Failed to receive hash");
00111     return false;
00112 }
00113
00114 logger.info("Received hash from client");
00115
00116 // Шаг 5: Аутентификация
00117 std::string combined = salt + password;
00118 std::string serverHash = Authenticator::computeSHA1(combined);
00119
00120 bool authResult = Authenticator::authenticate(password, salt, clientHash);
00121
00122
00123 if (authResult) {
00124     logger.info("Authentication successful for user: " + login);
00125     return sendMessage("OK");
00126 } else {
00127     logger.warning("Authentication failed for user: " + login);
00128     sendMessage("ERR");
00129     return false;
00130 }
00131 }
00132
00133 bool ClientHandler::receiveVectorsAndProcess() {
00134     // Шаг 6: Получаем количество векторов
00135     uint32_t numVectors;
00136     if (!receiveData(&numVectors, sizeof(numVectors))) {
00137         logger.error("Failed to receive number of vectors");
00138         return false;
00139     }
00140
00141     numVectors = le32toh(numVectors);
00142
00143     if (numVectors > 1000) {
00144         logger.error("Invalid number of vectors: " + std::to_string(numVectors));
00145         return false;
00146     }
00147
00148     // НЕ отправляем количество результатов - это не результат вычислений
00149     // Просто переходим к обработке векторов
00150
00151     // Обрабатываем каждый вектор
00152     for (uint32_t i = 0; i < numVectors; i++) {
00153         // Шаг 7: Получаем размер вектора
00154         uint32_t vectorSize;
00155         if (!receiveData(&vectorSize, sizeof(vectorSize))) {
00156             logger.error("Failed to receive vector size");
00157             return false;
00158         }
00159         vectorSize = le32toh(vectorSize);
00160
00161         if (vectorSize > 100000) {
00162             logger.error("Invalid vector size: " + std::to_string(vectorSize));
00163             return false;
00164         }
00165
00166         // Шаг 8: Получаем данные вектора
00167         std::vector<uint64_t> vector(vectorSize);
00168         for (uint32_t j = 0; j < vectorSize; j++) {
00169             uint64_t value;
00170             if (!receiveData(&value, sizeof(value))) {
00171                 logger.error("Failed to receive vector data");
00172                 return false;
00173             }
00174             value = le64toh(value);
00175             vector[j] = value;
00176         }
00177
00178         // Шаг 9: Вычисляем и отправляем результат
00179         uint64_t result = VectorCalculator::computeProduct(vector);
00180
00181         result = htobe64(result);
00182         if (!sendData(&result, sizeof(result))) {
00183             logger.error("Failed to send result");
00184             return false;
00185         }
00186     }
00187     return true;
00188 }
00189
00190 }
00191
00192 }
00193
00194 }
00195
00196 }
00197
00198 }
00199
00200 }
00201

```

```

00208 bool ClientHandler::sendSalt(const std::string& salt) {
00209     // Для соли отправляем БЕЗ нулевого байта (16 байт)
00210     return sendData(salt.c_str(), 16); // Только 16 байт
00211 }
00212
00219 bool ClientHandler::sendMessage(const std::string& message) {
00220     // Для текстовых сообщений отправляем С нулевым байтом
00221     return sendData(message.c_str(), message.length() + 1);
00222 }
00223
00231 bool ClientHandler::receiveData(void* buffer, size_t size) {
00232     // Для бинарных данных всегда читаем точное количество байт
00233     // без поиска нулевого терминатора
00234
00235     size_t totalReceived = 0;
00236     while (totalReceived < size) {
00237         ssize_t received = recv(clientSocket,
00238                                static_cast<char*>(buffer) + totalReceived,
00239                                size - totalReceived, 0);
00240
00241         if (received <= 0) {
00242             return false;
00243         }
00244         totalReceived += received;
00245     }
00246
00247     return true;
00248 }
00249
00257 bool ClientHandler::sendData(const void* data, size_t size) {
00258     const char* charData = static_cast<const char*>(data);
00259
00260     size_t totalSent = 0;
00261     while (totalSent < size) {
00262         ssize_t sent = send(clientSocket,
00263                             static_cast<const char*>(data) + totalSent,
00264                             size - totalSent, 0);
00265         if (sent <= 0) {
00266             return false;
00267         }
00268         totalSent += sent;
00269     }
00270     return true;
00271 }
00272
00283 bool ClientHandler::receiveString(std::string& result, size_t maxSize) {
00284     char buffer[256] = {0};
00285     size_t totalReceived = 0;
00286
00287     // Сначала посмотрим, что находится в буфере
00288     char peekBuffer[256] = {0};
00289     ssize_t peeked = recv(clientSocket, peekBuffer, sizeof(peekBuffer) - 1, MSG_PEEK);
00290
00291     if (peeked > 0) {
00292
00293         // Если мы видим полную строку без терминатора, считываем ровно столько байт
00294         bool hasTerminator = false;
00295         for (int i = 0; i < peeked; i++) {
00296             if (peekBuffer[i] == '\0' || peekBuffer[i] == '\n' || peekBuffer[i] == '\r') {
00297                 hasTerminator = true;
00298                 break;
00299             }
00300         }
00301     }
00302
00303     if (!hasTerminator && peeked > 0) {
00304         // Терминатор не найден, прочитаем ровно столько байт, сколько мы просмотрели
00305         ssize_t received = recv(clientSocket, buffer, peeked, 0);
00306         if (received == peeked) {
00307             buffer[received] = '\0'; // Добавляем нулевой терминатор
00308             result = std::string(buffer);
00309             return true;
00310         }
00311     }
00312 }
00313
00314 // Резервный вариант: читаем побайтно
00315 while (totalReceived < maxSize - 1) {
00316     ssize_t received = recv(clientSocket, &buffer[totalReceived], 1, 0);
00317
00318     if (received <= 0) {
00319         return false;
00320     }
00321
00322     // Проверяем наличие различных возможных терминаторов
00324     if (buffer[totalReceived] == '\0' ||

```

```

00325     buffer[totalReceived] == '\n' ||
00326     buffer[totalReceived] == '\r' ) {
00327
00328     buffer[totalReceived] = '\0'; // Гарантируем нулевой терминатор
00329     break;
00330 }
00331
00332 totalReceived++;
00333
00334 // Если мы получили 4 байта и это «пользователь», считаем, что он завершен
00335 if (totalReceived == 4 && strcmp(buffer, "user", 4) == 0) {
00336     buffer[4] = '\0'; // Добавляем нулевой терминатор
00337     break;
00338 }
00339 }
00340
00341 result = std::string(buffer);
00342 return true;
00343 }

```

4.11 Файл ClientHandler.h

```

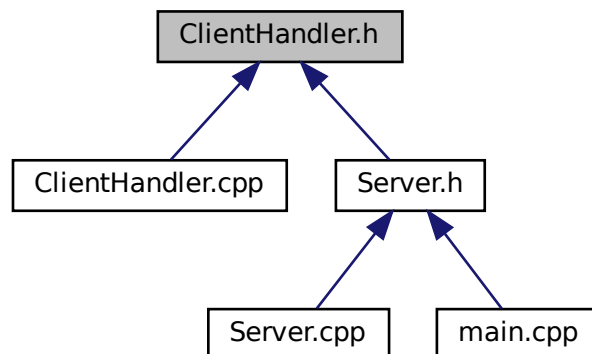
#include "ClientBase.h"
#include "Logger.h"
#include "Authenticator.h"
#include "VectorCalculator.h"
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <vector>

```

Граф включаемых заголовочных файлов для ClientHandler.h:



Граф файлов, в которые включается этот файл:



Классы

- class `ClientHandler`

Класс для обработки клиентских подключений

4.12 ClientHandler.h

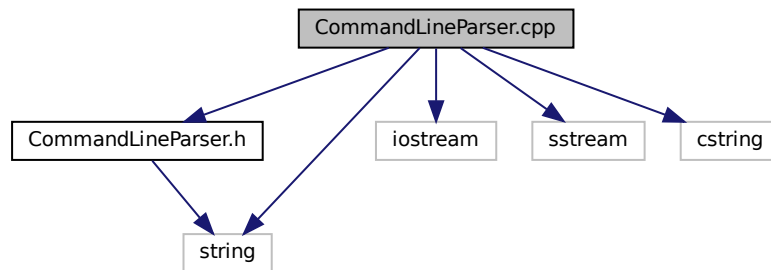
[См. документацию.](#)

```
00001 #ifndef CLIENTHANDLER_H
00002 #define CLIENTHANDLER_H
00003
00004 #include "ClientBase.h"
00005 #include "Logger.h"
00006 #include "Authenticator.h"
00007 #include "VectorCalculator.h"
00008 #include <sys/socket.h>
00009 #include <netinet/in.h>
00010 #include <unistd.h>
00011 #include <arpa/inet.h>
00012 #include <vector>
00013
00021 class ClientHandler {
00022 public:
00030     ClientHandler(int clientSocket, const sockaddr_in& clientAddr,
00031                  ClientBase& clientBase, Logger& logger);
00032
00036     void handle();
00037
00038 private:
00039     int clientSocket;
00040     sockaddr_in clientAddr;
00041     ClientBase& clientBase;
00042     Logger& logger;
00043
00048     bool authenticateClient();
00049
00054     bool receiveVectorsAndProcess();
00055
00061     bool sendMessage(const std::string& message);
00062
00069     bool receiveData(void* buffer, size_t size);
00070
00077     bool sendData(const void* data, size_t size);
00078
00085     bool receiveString(std::string& result, size_t maxSize = 256);
00086
00092     bool sendSalt(const std::string& salt);
00093 };
00094
00095 #endif
```

4.13 Файл CommandLineParser.cpp

```
#include "CommandLineParser.h"
#include <iostream>
#include <sstream>
#include <string>
#include <cstring>
```

Граф включаемых заголовочных файлов для CommandLineParser.cpp:



4.14 CommandLineParser.cpp

[См. документацию.](#)

```

00001 #include "CommandLineParser.h"
00002 #include <iostream>
00003 #include <sstream>
00004 #include <string>
00005 #include <cstring>
00006
00010 CommandLineParser::CommandLineParser() {
00011     config.port = 0;
00012     config.clientBaseFile = "";
00013     config.logFile = "";
00014
00015     std::ostringstream oss;
00016     oss << "Server usage:\n"
00017         << "  -b, --base <file>    Client database file (REQUIRED)\n"
00018         << "  -l, --log <file>       Log file (REQUIRED)\n"
00019         << "  -p, --port <port>     Port to listen on (REQUIRED)\n"
00020         << "  -h, --help           Show this help\n"
00021         << "Example: ./server -b clients.txt -l server.log -p 33333\n";
00022     helpText = oss.str();
00023 }
00024
00031 bool CommandLineParser::parse(int argc, char* argv[]) {
00032     // Всегда сбрасываем конфиг перед парсингом
00033     config.port = 0;
00034     config.clientBaseFile = "";
00035     config.logFile = "";
00036
00037     // Ручной парсинг аргументов
00038     for (int i = 1; i < argc; i++) {
00039         std::string arg = argv[i];
00040
00041         if (arg == "-h" || arg == "--help") {
00042             std::cout << helpText;
00043             return false;
00044         }
00045         else if (arg == "-b" || arg == "--base") {
00046             if (i + 1 < argc) {
00047                 config.clientBaseFile = argv[++i];
00048             } else {
00049                 std::cerr << "Error: Missing value for " << arg << std::endl;
00050                 return false;
00051             }
00052         }
00053         else if (arg == "-l" || arg == "--log") {
00054             if (i + 1 < argc) {
00055                 config.logFile = argv[++i];
00056             } else {
00057                 std::cerr << "Error: Missing value for " << arg << std::endl;
00058                 return false;
00059             }
00060         }
00061         else if (arg == "-p" || arg == "--port") {
00062             if (i + 1 < argc) {

```

```

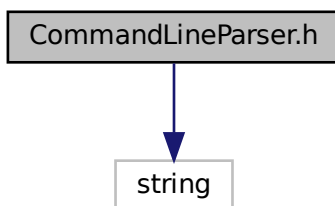
00063         try {
00064             config.port = std::stoi(argv[++i]);
00065         } catch (const std::exception&) {
00066             std::cerr << "Error: Invalid port number" << std::endl;
00067             return false;
00068         }
00069     } else {
00070         std::cerr << "Error: Missing value for " << arg << std::endl;
00071         return false;
00072     }
00073 }
00074 else {
00075     std::cerr << "Error: Unknown option: " << arg << std::endl;
00076     return false;
00077 }
00078 }
00079
00080 // Проверяем обязательные параметры
00081 if (config.clientBaseFile.empty() || config.logFile.empty() || config.port == 0) {
00082     std::cerr << "Error: All parameters are required!\n\n";
00083     std::cerr << helpText;
00084     return false;
00085 }
00086
00087 // Проверяем значение порта
00088 if (config.port < 1 || config.port > 65535) {
00089     std::cerr << "Error: Port must be between 1 and 65535\n";
00090     return false;
00091 }
00092
00093 return true;
00094 }
00095
00100 std::string CommandLineParser::getHelp() const {
00101     return helpText;
00102 }

```

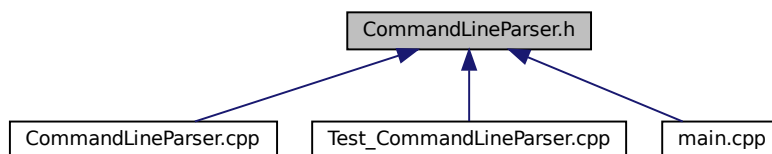
4.15 Файл CommandLineParser.h

#include <string>

Граф включаемых заголовочных файлов для CommandLineParser.h:



Граф файлов, в которые включается этот файл:



Классы

- struct [ServerConfig](#)
Структура для хранения конфигурации сервера
- class [CommandLineParser](#)
Класс для разбора командной строки

4.16 CommandLineParser.h

[См. документацию.](#)

```

00001 #ifndef COMMANDLINEPARSER_H
00002 #define COMMANDLINEPARSER_H
00003
00004 #include <string>
00005
00009 struct ServerConfig {
00010     std::string clientBaseFile;
00011     std::string logFile;
00012     int port;
00013 };
00014
00022 class CommandLineParser {
00023 public:
00027     CommandLineParser();
00028
00035     bool parse(int argc, char* argv[]);
00036
00041     ServerConfig getConfig()const { return config; }
00042
00047     std::string getHelp() const;
00048
00049 private:
00050     ServerConfig config;
00051     std::string helpText;
00052 };
00053
00054 #endif
  
```

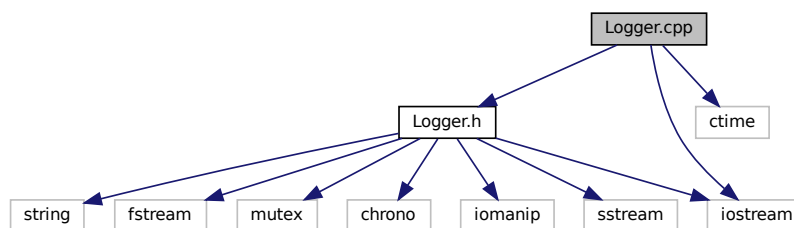
4.17 Файл Logger.cpp

```

#include "Logger.h"
#include <iostream>
  
```

```
#include <ctime>
```

Граф включаемых заголовочных файлов для Logger.cpp:



4.18 Logger.cpp

[См. документацию.](#)

```

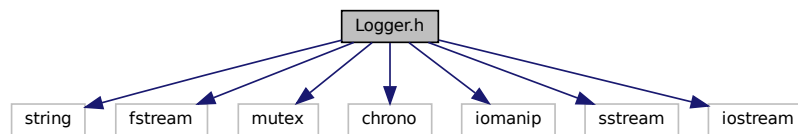
00001 #include "Logger.h"
00002 #include <iostream>
00003 #include <ctime>
00004
00010 Logger::Logger(const std::string& filename) {
00011     logFile.open(filename, std::ios::app);
00012     if (!logFile.is_open()) {
00013         throw std::runtime_error("Cannot open log file: " + filename);
00014     }
00015 }
00016
00020 Logger::~Logger() {
00021     if (logFile.is_open()) {
00022         logFile.close();
00023     }
00024 }
00025
00031 void Logger::log(LogLevel level, const std::string& message) {
00032     std::lock_guard<std::mutex> lock(logMutex);
00033
00034     std::string logEntry = getCurrentTime() + " [" + levelToString(level) + "] " + message + "\n";
00035
00036     logFile << logEntry;
00037     logFile.flush();
00038
00039     // Also output to console
00040     std::cout << logEntry;
00041 }
00042
00047 std::string Logger::getCurrentTime() {
00048     auto now = std::chrono::system_clock::now();
00049     auto time_t = std::chrono::system_clock::to_time_t(now);
00050
00051     std::stringstream ss;
00052
00053     // Безопасная альтернатива: используем статический буфер с мьютексом
00054     static std::mutex time_mutex;
00055     std::tm tm_buffer;
00056
00057     {
00058         std::lock_guard<std::mutex> lock(time_mutex);
00059         tm_buffer = *std::localtime(&time_t);
00060     }
00061
00062     ss << std::put_time(&tm_buffer, "%Y-%m-%d %H:%M:%S");
00063     return ss.str();
00064 }
00065
00071 std::string Logger::levelToString(LogLevel level) {
00072     switch (level) {
00073         case LogLevel::INFO: return "INFO";
00074         case LogLevel::WARNING: return "WARNING";
00075         case LogLevel::ERROR: return "ERROR";
00076         default: return "UNKNOWN";
00077     }
00078 }

```

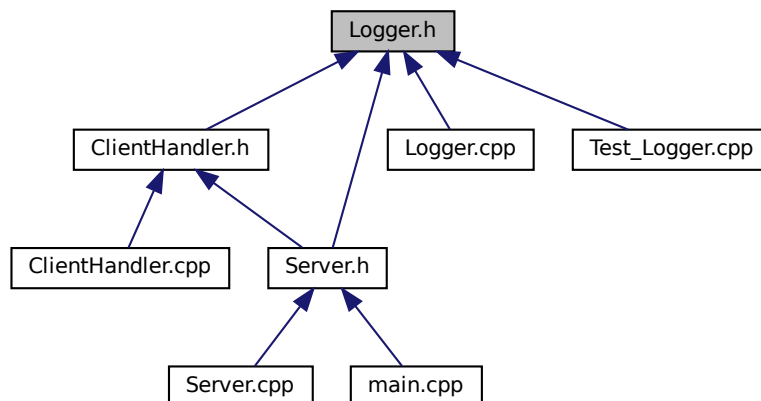
4.19 Файл Logger.h

```
#include <string>
#include <fstream>
#include <mutex>
#include <chrono>
#include <iomanip>
#include <sstream>
#include <iostream>
```

Граф включаемых заголовочных файлов для Logger.h:



Граф файлов, в которые включается этот файл:



Классы

- class `Logger`
Класс для логирования событий сервера

Перечисления

- enum class `LogLevel` { `INFO` , `WARNING` , `ERROR` }
Перечисление уровней логирования

4.19.1 Перечисления

4.19.1.1 LogLevel

```
enum class LogLevel [strong]
```

Перечисление уровней логирования

Элементы перечислений

INFO	Информационное сообщение
WARNING	Предупреждение
ERROR	Ошибка

См. определение в файле [Logger.h](#) строка 15

4.20 Logger.h

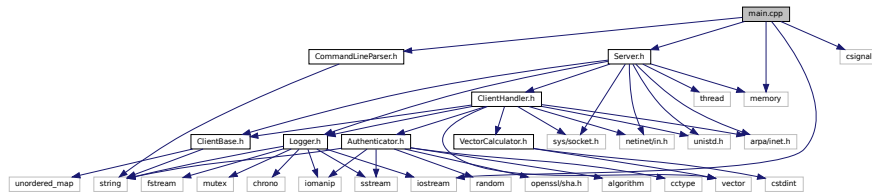
[См. документацию.](#)

```
00001 #ifndef LOGGER_H
00002 #define LOGGER_H
00003
00004 #include <string>
00005 #include <fstream>
00006 #include <mutex>
00007 #include <chrono>
00008 #include <iomanip>
00009 #include <sstream>
00010 #include <iostream>
00011
00015 enum class LogLevel {
00016     INFO,
00017     WARNING,
00018     ERROR
00019 };
00020
00028 class Logger {
00029 public:
00035     Logger(const std::string& filename);
00036
00040     ~Logger();
00041
00047     void log(LogLevel level, const std::string& message);
00048
00053     void info(const std::string& message) { log(LogLevel::INFO, message); }
00054
00059     void warning(const std::string& message) { log(LogLevel::WARNING, message); }
00060
00065     void error(const std::string& message) { log(LogLevel::ERROR, message); }
00066
00067 private:
00068     std::ofstream logFile;
00069     std::mutex logMutex;
00070
00075     std::string getCurrentTime();
00076
00082     std::string levelToString(LogLevel level);
00083 };
00084
00085 #endif
```

4.21 Файл main.cpp

```
#include "CommandLineParser.h"
#include "Server.h"
#include <iostream>
#include <csignal>
#include <memory>
```

Граф включаемых заголовочных файлов для main.cpp:



Функции

- void [signalHandler](#) (int signal)
Обработчик сигналов для graceful shutdown.
- int [main](#) (int argc, char *argv[])
Главная функция сервера

Переменные

- std::unique_ptr< [Server](#) > [server](#)
Глобальный указатель на сервер для обработки сигналов

4.21.1 Функции

4.21.1.1 main()

```
int main (
    int argc,
    char * argv[] )
```

Главная функция сервера

Аргументы

argc	Количество аргументов командной строки
argv	Массив аргументов командной строки

Возвращает

Код завершения программы

Автор

Гришин Николай

Версия

1.0

Дата

24.11.2025

Авторство

ИБСТ ПГУ

См. определение в файле [main.cpp](#) строка [30](#)

4.21.1.2 signalHandler()

```
void signalHandler (  
    int signal )
```

Обработчик сигналов для graceful shutdown.

Аргументы

signal	Номер сигнала
--------	---------------

См. определение в файле [main.cpp](#) строка [13](#)

4.21.2 Переменные

4.21.2.1 server

```
std::unique_ptr<Server> server
```

Глобальный указатель на сервер для обработки сигналов

См. определение в файле [main.cpp](#) строка [7](#)

4.22 main.cpp

См. документацию.

```

00001 #include "CommandLineParser.h"
00002 #include "Server.h"
00003 #include <iostream>
00004 #include <signal>
00005 #include <memory>
00006
00007 std::unique_ptr<Server> server;
00008
00013 void signalHandler(int signal) {
00014     if (server) {
00015         std::cout << "\nShutting down server..." << std::endl;
00016         server->stop();
00017     }
00018 }
00019
00030 int main(int argc, char* argv[]) {
00031     // Настройка обработчиков сигналов
00032     signal(SIGINT, signalHandler);
00033     signal(SIGTERM, signalHandler);
00034
00035     // Разбор аргументов командной строки
00036     CommandLineParser parser;
00037     if (!parser.parse(argc, argv)) {
00038         return 0;
00039     }
00040
00041     ServerConfig config = parser.getConfig();
00042
00043     try {
00044         // Создание и инициализация сервера
00045         server = std::make_unique<Server>(config.clientBaseFile, config.logFile, config.port);
00046
00047         if (!server->initialize()) {
00048             std::cerr << "Failed to initialize server" << std::endl;
00049             return 1;
00050         }
00051
00052         // Запуск сервера
00053         server->run();
00054
00055     } catch (const std::exception& e) {
00056         std::cerr << "Server error: " << e.what() << std::endl;
00057         return 1;
00058     }
00059
00060     return 0;
00061 }

```

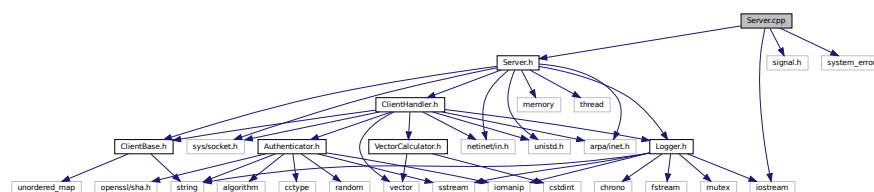
4.23 Файл Server.cpp

```

#include "Server.h"
#include <iostream>
#include <signal.h>
#include <system_error>

```

Граф включаемых заголовочных файлов для Server.cpp:



4.24 Server.cpp

[См. документацию.](#)

```

00001 #include "Server.h"
00002 #include <iostream>
00003 #include <signal.h>
00004 #include <system_error>
00005
00012 Server::Server(const std::string& clientBaseFile, const std::string& logFile, int port)
00013     : clientBaseFile(clientBaseFile), logFile(logFile), port(port),
00014       serverSocket(-1), running(false) {}
00015
00019 Server::~Server() {
00020     cleanup();
00021 }
00022
00027 bool Server::initialize() {
00028     // Инициализация базы клиентов
00029     clientBase = std::make_unique<ClientBase>(clientBaseFile);
00030     if (!clientBase->load()) {
00031         std::cerr << "Failed to load client database: " << clientBaseFile << std::endl;
00032         return false;
00033     }
00034
00035     // ДОБАВИТЬ: проверим что база загрузилась
00036     std::cout << "DEBUG: Client database loaded successfully" << std::endl;
00037
00038     // Инициализация Логера
00039     try {
00040         logger = std::make_unique<Logger>(logFile);
00041     } catch (const std::exception& e) {
00042         std::cerr << "Failed to initialize logger: " << e.what() << std::endl;
00043         return false;
00044     }
00045
00046     // Создание сокета
00047     serverSocket = socket(AF_INET, SOCK_STREAM, 0);
00048     if (serverSocket == -1) {
00049         logger->error("Failed to create socket");
00050         return false;
00051     }
00052
00053     // Установка параметров сокета
00054     int opt = 1;
00055     if (setsockopt(serverSocket, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) == -1) {
00056         logger->error("Failed to set socket options");
00057         return false;
00058     }
00059
00060     // Привязка сокета
00061     sockaddr_in serverAddr{};
00062     serverAddr.sin_family = AF_INET;
00063     serverAddr.sin_addr.s_addr = INADDR_ANY;
00064     serverAddr.sin_port = htons(port);
00065
00066     if (bind(serverSocket, (struct sockaddr*)&serverAddr, sizeof(serverAddr)) == -1) {
00067         logger->error("Failed to bind socket to port " + std::to_string(port));
00068         return false;
00069     }
00070
00071     // Listen for connections
00072     if (listen(serverSocket, 10) == -1) {
00073         logger->error("Failed to listen on socket");
00074         return false;
00075     }
00076
00077     logger->info("Server initialized successfully on port " + std::to_string(port));
00078     return true;
00079 }
00080
00084 void Server::run() {
00085     running = true;
00086     logger->info("Server started and listening for connections");
00087
00088     while (running) {
00089         sockaddr_in clientAddr{};
00090         socklen_t clientAddrLen = sizeof(clientAddr);
00091
00092         int clientSocket = accept(serverSocket, (struct sockaddr*)&clientAddr, &clientAddrLen);
00093         if (clientSocket == -1) {
00094             if (running) {
00095                 logger->error("Failed to accept client connection");
00096             }
00097             continue;
00098         }
    
```

```

00099
00100     ClientHandler handler(clientSocket, clientAddr, *clientBase, *logger);
00101     handler.handle();
00102 }
00103 }
00104
00108 void Server::stop() {
00109     running = false;
00110     if (serverSocket != -1) {
00111         close(serverSocket);
00112         serverSocket = -1;
00113     }
00114     logger->info("Server stopped");
00115 }
00116
00120 void Server::cleanup() {
00121     stop();
00122 }

```

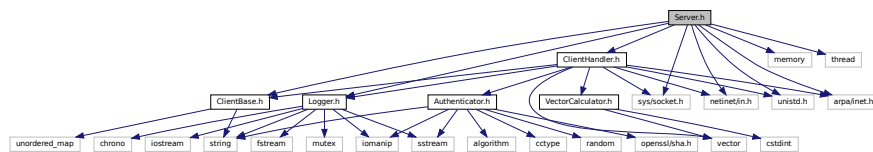
4.25 Файл Server.h

```

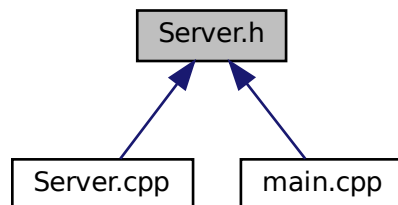
#include "ClientBase.h"
#include "Logger.h"
#include "ClientHandler.h"
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <memory>
#include <thread>

```

Граф включаемых заголовочных файлов для Server.h:



Граф файлов, в которые включается этот файл:



Классы

- class `Server`

Основной класс сервера

4.26 Server.h

[См. документацию.](#)

```

00001 #ifndef SERVER_H
00002 #define SERVER_H
00003
00004 #include "ClientBase.h"
00005 #include "Logger.h"
00006 #include "ClientHandler.h"
00007 #include <sys/socket.h>
00008 #include <netinet/in.h>
00009 #include <unistd.h>
00010 #include <arpa/inet.h>
00011 #include <memory>
00012 #include <thread>
00013
00021 class Server {
00022 public:
00029     Server(const std::string& clientBaseFile, const std::string& logFile, int port);
00030
00034     ~Server();
00035
00040     bool initialize();
00041
00045     void run();
00046
00050     void stop();
00051
00052 private:
00053     std::string clientBaseFile;
00054     std::string logFile;
00055     int port;
00056     int serverSocket;
00057     bool running;
00058
00059     std::unique_ptr<ClientBase> clientBase;
00060     std::unique_ptr<Logger> logger;
00061
00065     void cleanup();
00066 };
00067
00068 #endif

```

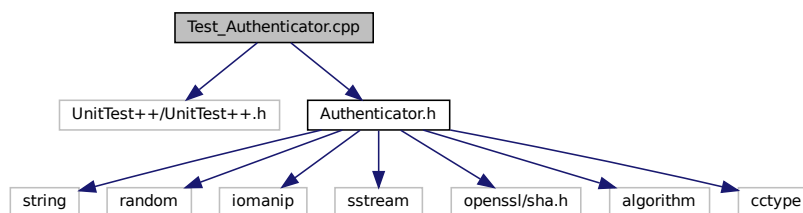
4.27 Файл Test_Authenticator.cpp

```

#include <UnitTest++/UnitTest++.h>
#include "Authenticator.h"

```

Граф включаемых заголовочных файлов для Test_Authenticator.cpp:



Функции

- [SUITE](#) (AuthenticatorTest)

4.27.1 Функции

4.27.1.1 SUITE()

```
SUITE (
    AuthenticatorTest )
```

См. определение в файле [Test_Authenticator.cpp](#) строка 4

4.28 Test_Authenticator.cpp

См. документацию.

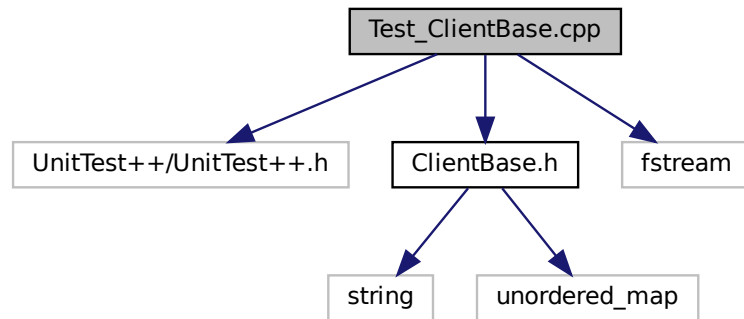
```
00001 #include <UnitTest++/UnitTest++.h>
00002 #include "Authenticator.h"
00003
00004 SUITE(AuthenticatorTest)
00005 {
00006     TEST(GenerateSalt) {
00007         std::string salt = Authenticator::generateSalt();
00008
00009         CHECK_EQUAL(16, salt.length()); // 16 hex digits
00010         for (char c : salt) {
00011             CHECK(isxdigit(c)); // All characters should be hex digits
00012         }
00013     }
00014
00015     TEST(ComputeSHA1) {
00016         std::string testData = "test123";
00017         std::string hash = Authenticator::computeSHA1(testData);
00018
00019         CHECK_EQUAL(40, hash.length()); // SHA-1 produces 40 hex chars
00020         for (char c : hash) {
00021             CHECK(isxdigit(c));
00022         }
00023     }
00024
00025     TEST(AuthenticationSuccess) {
00026         std::string password = "P@ssW0rd";
00027         std::string salt = "1234567890ABCDEF";
00028
00029         std::string combined = salt + password;
00030         std::string clientHash = Authenticator::computeSHA1(combined);
00031
00032         bool result = Authenticator::authenticate(password, salt, clientHash);
00033         CHECK(result);
00034     }
00035
00036     TEST(AuthenticationFailure) {
00037         std::string password = "P@ssW0rd";
00038         std::string salt = "1234567890ABCDEF";
00039         std::string wrongHash = "invalidhash1234567890abcdef1234567890abcdef";
00040
00041         bool result = Authenticator::authenticate(password, salt, wrongHash);
00042         CHECK(!result);
00043     }
00044 }
```

4.29 Файл Test_ClientBase.cpp

```
#include <UnitTest++/UnitTest++.h>
#include "ClientBase.h"
```

```
#include <fstream>
```

Граф включаемых заголовочных файлов для Test_ClientBase.cpp:



Классы

- struct [ClientBaseFixture](#)

Функции

- [SUITE](#) (ClientBaseTest)

4.29.1 Функции

4.29.1.1 SUITE()

```
SUITE (
    ClientBaseTest )
```

См. определение в файле [Test_ClientBase.cpp](#) строка [23](#)

4.30 Test_ClientBase.cpp

[См. документацию.](#)

```

00001 #include <UnitTest++/UnitTest++.h>
00002 #include "ClientBase.h"
00003 #include <fstream>
00004
00005 struct ClientBaseFixture {
00006     std::string testFilename = "test_clients.txt";
00007
00008     ClientBaseFixture() {
00009         // Create test file
00010         std::ofstream file(testFilename);
00011         file << "user1:pass1\n";
  
```

```

00012     file << "user2:pass2\n";
00013     file << "user3:pass3\n";
00014     file.close();
00015 }
00016
00017 ~ClientBaseFixture() {
00018     // Remove test file
00019     remove(testFilename.c_str());
00020 }
00021 };
00022
00023 SUITE(ClientBaseTest)
00024 {
00025     TEST_FIXTURE(ClientBaseFixture, LoadSuccess) {
00026         ClientBase clientBase(testFilename);
00027         CHECK(clientBase.load());
00028     }
00029
00030     TEST_FIXTURE(ClientBaseFixture, UserExists) {
00031         ClientBase clientBase(testFilename);
00032         clientBase.load();
00033
00034         CHECK(clientBase.userExists("user1"));
00035         CHECK(clientBase.userExists("user2"));
00036         CHECK(!clientBase.userExists("nonexistent"));
00037     }
00038
00039     TEST_FIXTURE(ClientBaseFixture, GetPassword) {
00040         ClientBase clientBase(testFilename);
00041         clientBase.load();
00042
00043         CHECK_EQUAL("pass1", clientBase.getPassword("user1"));
00044         CHECK_EQUAL("pass2", clientBase.getPassword("user2"));
00045         CHECK_EQUAL("", clientBase.getPassword("nonexistent"));
00046     }
00047
00048     TEST(LoadFailure) {
00049         ClientBase clientBase("nonexistent_file.txt");
00050         CHECK(!clientBase.load());
00051     }
00052 }

```

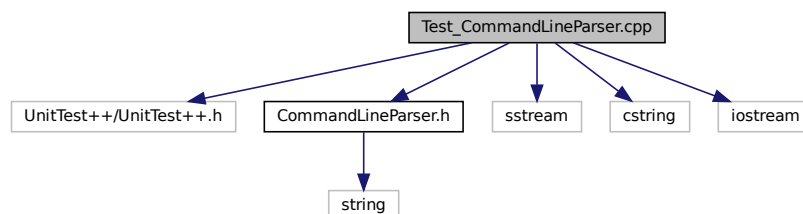
4.31 Файл Test_CommandLineParser.cpp

```

#include <UnitTest++/UnitTest++.h>
#include "CommandLineParser.h"
#include <sstream>
#include <cstring>
#include <iostream>

```

Граф включаемых заголовочных файлов для Test_CommandLineParser.cpp:



Классы

- class [OutputRedirect](#)

Функции

- [SUITE](#) (CommandLineParserTest)

4.31.1 Функции

4.31.1.1 SUITE()

SUITE (
 CommandLineParserTest)

См. определение в файле [Test_CommandLineParser.cpp](#) строка 33

4.32 Test_CommandLineParser.cpp

[См. документацию.](#)

```
00001 #include <UnitTest++/UnitTest++.h>
00002 #include "CommandLineParser.h"
00003 #include <sstream>
00004 #include <cstring>
00005 #include <iostream>
00006
00007 class OutputRedirect {
00008     std::stringstream buffer;
00009     std::streambuf* old_cout;
00010     std::streambuf* old_cerr;
00011
00012 public:
00013     OutputRedirect() {
00014         old_cout = std::cout.rdbuf(buffer.rdbuf());
00015         old_cerr = std::cerr.rdbuf(buffer.rdbuf());
00016     }
00017
00018     ~OutputRedirect() {
00019         std::cout.rdbuf(old_cout);
00020         std::cerr.rdbuf(old_cerr);
00021     }
00022
00023     std::string getOutput()const {
00024         return buffer.str();
00025     }
00026
00027     void clear() {
00028         buffer.str("");
00029         buffer.clear();
00030     }
00031 };
00032
00033 SUITE(CommandLineParserTest)
00034 {
00035     TEST(ValidParameters) {
00036         OutputRedirect redirect;
00037         CommandLineParser parser;
00038
00039         const char* argv[] = {"server", "-b", "clients.txt", "-l", "server.log", "-p", "33333", nullptr};
00040         int argc = sizeof(argv) / sizeof(argv[0]) - 1;
00041
00042         CHECK(parser.parse(argc, const_cast<char*>(argv)));
00043
00044         ServerConfig config = parser.getConfig();
00045         CHECK_EQUAL("clients.txt", config.clientBaseFile);
00046         CHECK_EQUAL("server.log", config.logFile);
00047         CHECK_EQUAL(33333, config.port);
00048     }
00049
00050     TEST(HelpParameter) {
00051         OutputRedirect redirect;
00052         CommandLineParser parser;
```

```

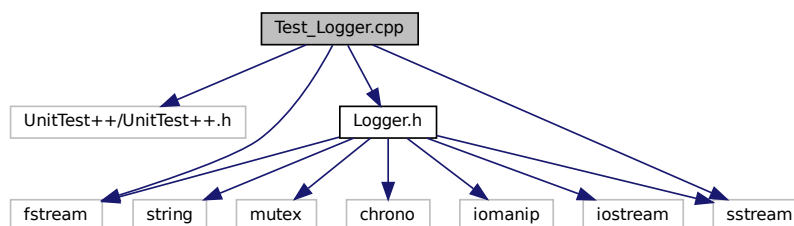
00053
00054     const char* argv[] = {"server", "-h", nullptr};
00055     int argc = sizeof(argv) / sizeof(argv[0]) - 1;
00056
00057     // parse должен вернуть false для -h, но без исключений
00058     bool result = parser.parse(argc, const_cast<char*>(argv));
00059     CHECK(!result);
00060
00061     // Проверяем что вывод содержит справку
00062     std::string output = redirect.getOutput();
00063     CHECK(output.find("Server usage") != std::string::npos);
00064 }
00065
00066 TEST(MissingRequiredParameters) {
00067     OutputRedirect redirect;
00068     CommandLineParser parser;
00069
00070     const char* argv[] = {"server", "-b", "clients.txt", nullptr};
00071     int argc = sizeof(argv) / sizeof(argv[0]) - 1;
00072
00073     bool result = parser.parse(argc, const_cast<char*>(argv));
00074     CHECK(!result);
00075
00076     std::string output = redirect.getOutput();
00077     CHECK(output.find("Error: All parameters are required") != std::string::npos);
00078 }
00079
00080 TEST(InvalidPort) {
00081     OutputRedirect redirect;
00082     CommandLineParser parser;
00083
00084     const char* argv[] = {"server", "-b", "clients.txt", "-l", "server.log", "-p", "70000", nullptr};
00085     int argc = sizeof(argv) / sizeof(argv[0]) - 1;
00086
00087     bool result = parser.parse(argc, const_cast<char*>(argv));
00088     CHECK(!result);
00089
00090     std::string output = redirect.getOutput();
00091     CHECK(output.find("Port must be between 1 and 65535") != std::string::npos);
00092 }
00093
00094 TEST(LongOptions) {
00095     OutputRedirect redirect;
00096     CommandLineParser parser;
00097
00098     const char* argv[] = {"server", "--base", "clients.txt", "--log", "server.log", "--port", "44444", nullptr};
00099     int argc = sizeof(argv) / sizeof(argv[0]) - 1;
00100
00101     CHECK(parser.parse(argc, const_cast<char*>(argv)));
00102
00103     ServerConfig config = parser.getConfig();
00104     CHECK_EQUAL("clients.txt", config.clientBaseFile);
00105     CHECK_EQUAL("server.log", config.logFile);
00106     CHECK_EQUAL(44444, config.port);
00107 }
00108
00109 TEST(MixedOptions) {
00110     OutputRedirect redirect;
00111     CommandLineParser parser;
00112
00113     const char* argv[] = {"server", "-b", "clients.txt", "--log", "server.log", "-p", "55555", nullptr};
00114     int argc = sizeof(argv) / sizeof(argv[0]) - 1;
00115
00116     CHECK(parser.parse(argc, const_cast<char*>(argv)));
00117
00118     ServerConfig config = parser.getConfig();
00119     CHECK_EQUAL("clients.txt", config.clientBaseFile);
00120     CHECK_EQUAL("server.log", config.logFile);
00121     CHECK_EQUAL(55555, config.port);
00122 }
00123
00124 TEST(InvalidPortFormat) {
00125     OutputRedirect redirect;
00126     CommandLineParser parser;
00127
00128     const char* argv[] = {"server", "-b", "clients.txt", "-l", "server.log", "-p", "not_a_number", nullptr};
00129     int argc = sizeof(argv) / sizeof(argv[0]) - 1;
00130
00131     bool result = parser.parse(argc, const_cast<char*>(argv));
00132     CHECK(!result);
00133
00134     std::string output = redirect.getOutput();
00135     CHECK(output.find("Invalid port number") != std::string::npos);
00136 }
00137 }

```

4.33 Файл Test_Logger.cpp

```
#include <UnitTest++/UnitTest++.h>
#include "Logger.h"
#include <fstream>
#include <sstream>
```

Граф включаемых заголовочных файлов для Test_Logger.cpp:



Классы

- struct [LoggerFixture](#)

Функции

- [SUITE](#) (LoggerTest)

4.33.1 Функции

4.33.1.1 SUITE()

```
SUITE (
    LoggerTest )
```

См. определение в файле [Test_Logger.cpp](#) строка [25](#)

4.34 Test_Logger.cpp

См. документацию.

```

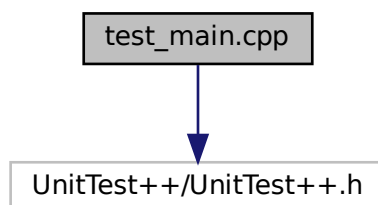
00001 #include <UnitTest++/UnitTest++.h>
00002 #include "Logger.h"
00003 #include <fstream>
00004 #include <sstream>
00005
00006 struct LoggerFixture {
00007     std::string testLogFile = "test_log.txt";
00008
00009     LoggerFixture() {
00010         remove(testLogFile.c_str());
00011     }
00012
00013     ~LoggerFixture() {
00014         remove(testLogFile.c_str());
00015     }
00016
00017     std::string readLogFile() {
00018         std::ifstream file(testLogFile);
00019         std::stringstream buffer;
00020         buffer << file.rdbuf();
00021         return buffer.str();
00022     }
00023 };
00024
00025 SUITE(LoggerTest)
00026 {
00027     TEST_FIXTURE(LoggerFixture, Constructor) {
00028         bool success = false;
00029         try {
00030             Logger logger(testLogFile);
00031             success = true;
00032         } catch (...) {
00033             success = false;
00034         }
00035         CHECK(success);
00036     }
00037
00038     TEST_FIXTURE(LoggerFixture, LogInfo) {
00039         Logger logger(testLogFile);
00040         logger.info("Test info message");
00041
00042         std::string logContent = readLogFile();
00043         CHECK(logContent.find("INFO") != std::string::npos);
00044         CHECK(logContent.find("Test info message") != std::string::npos);
00045     }
00046
00047     TEST_FIXTURE(LoggerFixture, LogWarning) {
00048         Logger logger(testLogFile);
00049         logger.warning("Test warning message");
00050
00051         std::string logContent = readLogFile();
00052         CHECK(logContent.find("WARNING") != std::string::npos);
00053         CHECK(logContent.find("Test warning message") != std::string::npos);
00054     }
00055
00056     TEST_FIXTURE(LoggerFixture, LogError) {
00057         Logger logger(testLogFile);
00058         logger.error("Test error message");
00059
00060         std::string logContent = readLogFile();
00061         CHECK(logContent.find("ERROR") != std::string::npos);
00062         CHECK(logContent.find("Test error message") != std::string::npos);
00063     }
00064
00065     TEST_FIXTURE(LoggerFixture, MultipleLogs) {
00066         Logger logger(testLogFile);
00067         logger.info("Message 1");
00068         logger.warning("Message 2");
00069         logger.error("Message 3");
00070
00071         std::string logContent = readLogFile();
00072         CHECK(logContent.find("Message 1") != std::string::npos);
00073         CHECK(logContent.find("Message 2") != std::string::npos);
00074         CHECK(logContent.find("Message 3") != std::string::npos);
00075     }
00076 }

```

4.35 Файл test_main.cpp

```
#include <UnitTest++/UnitTest++.h>
```

Граф включаемых заголовочных файлов для test_main.cpp:



Функции

- `int main (int argc, char **argv)`

4.35.1 Функции

4.35.1.1 main()

```
int main (  
    int argc,  
    char ** argv )
```

См. определение в файле `test_main.cpp` строка 4

4.36 test_main.cpp

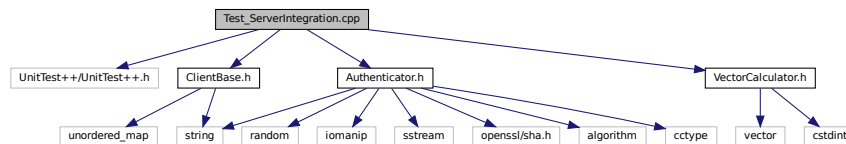
[См. документацию.](#)

```
00001 // test_main.cpp  
00002 #include <UnitTest++/UnitTest++.h>  
00003  
00004 int main(int argc, char** argv) {  
00005     return UnitTest::RunAllTests();  
00006 }
```

4.37 Файл Test_ServerIntegration.cpp

```
#include <UnitTest++/UnitTest++.h>
#include "ClientBase.h"
#include "Authenticator.h"
#include "VectorCalculator.h"
```

Граф включаемых заголовочных файлов для Test_ServerIntegration.cpp:



Функции

- [SUITE](#) (ServerIntegrationTest)

4.37.1 Функции

4.37.1.1 SUITE()

```
SUITE (
    ServerIntegrationTest )
```

См. определение в файле [Test_ServerIntegration.cpp](#) строка 6

4.38 Test_ServerIntegration.cpp

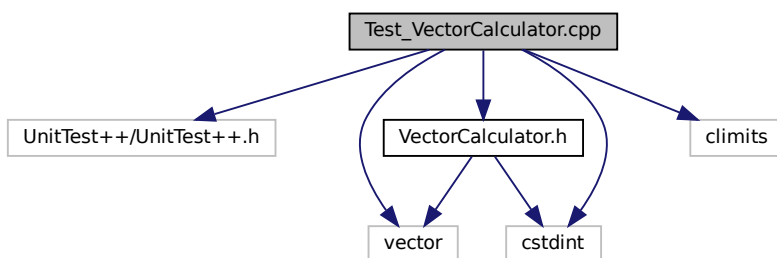
[См. документацию.](#)

```
00001 #include <UnitTest++/UnitTest++.h>
00002 #include "ClientBase.h"
00003 #include "Authenticator.h"
00004 #include "VectorCalculator.h"
00005
00006 SUITE(ServerIntegrationTest)
00007 {
00008     TEST(FullAuthenticationFlow) {
00009         // Setup
00010         std::string login = "testuser";
00011         std::string password = "testpass";
00012         std::string salt = Authenticator::generateSalt();
00013
00014         // Client side: compute hash
00015         std::string combined = salt + password;
00016         std::string clientHash = Authenticator::computeSHA1(combined);
00017
00018         // Server side: verify
00019         bool authResult = Authenticator::authenticate(password, salt, clientHash);
00020         CHECK(authResult);
00021     }
00022
00023     TEST(VectorProcessingIntegration) {
00024         // Test data
00025         std::vector<uint64_t> testVector = {2, 3, 5, 7};
00026
00027         // Process vector
00028         uint64_t result = VectorCalculator::computeProduct(testVector);
00029
00030         // Verify result
00031         CHECK_EQUAL(210, result); // 2*3*5*7 = 210
00032     }
00033 }
```

4.39 Файл Test_VectorCalculator.cpp

```
#include <UnitTest++/UnitTest++.h>
#include "VectorCalculator.h"
#include <vector>
#include <cstdint>
#include <climits>
```

Граф включаемых заголовочных файлов для Test_VectorCalculator.cpp:



Функции

- [SUITE](#) (VectorCalculatorTest)

4.39.1 Функции

4.39.1.1 SUITE()

```
SUITE (
    VectorCalculatorTest )
```

См. определение в файле [Test_VectorCalculator.cpp](#) строка 7

4.40 Test_VectorCalculator.cpp

[См. документацию.](#)

```
00001 #include <UnitTest++/UnitTest++.h>
00002 #include "VectorCalculator.h"
00003 #include <vector>
00004 #include <cstdint>
00005 #include <climits>
00006
00007 SUITE(VectorCalculatorTest)
00008 {
00009     TEST(EmptyVector) {
00010         std::vector<uint64_t> emptyVec;
00011         uint64_t result = VectorCalculator::computeProduct(emptyVec);
00012         CHECK_EQUAL(0, result);
00013     }
```

```

00014
00015     TEST(SingleElement) {
00016         std::vector<uint64_t> vec = {42};
00017         uint64_t result = VectorCalculator::computeProduct(vec);
00018         CHECK_EQUAL(42, result);
00019     }
00020
00021     TEST(MultipleElements) {
00022         std::vector<uint64_t> vec = {2, 3, 4};
00023         uint64_t result = VectorCalculator::computeProduct(vec);
00024         CHECK_EQUAL(24, result);
00025     }
00026
00027     TEST(WithZero) {
00028         std::vector<uint64_t> vec = {1, 2, 0, 4};
00029         uint64_t result = VectorCalculator::computeProduct(vec);
00030         CHECK_EQUAL(0, result);
00031     }
00032
00033     TEST(OverflowDetection) {
00034         std::vector<uint64_t> vec = {UINT64_MAX, 2};
00035         uint64_t result = VectorCalculator::computeProduct(vec);
00036         CHECK_EQUAL(UINT64_MAX, result);
00037     }
00038
00039     TEST(LargeNumbers) {
00040         std::vector<uint64_t> vec = {1000000, 1000000};
00041         uint64_t result = VectorCalculator::computeProduct(vec);
00042         CHECK_EQUAL(1000000000000, result);
00043     }
00044 }

```

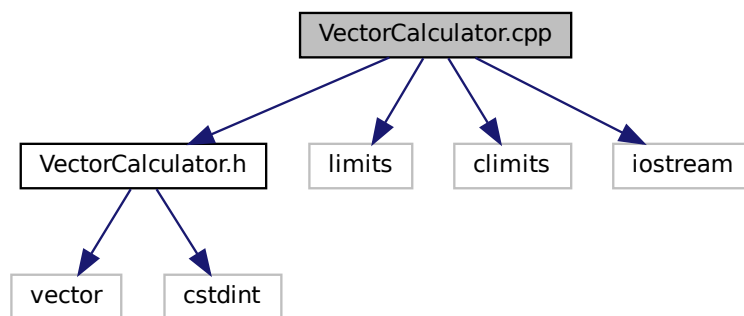
4.41 Файл VectorCalculator.cpp

```

#include "VectorCalculator.h"
#include <limits>
#include <climits>
#include <iostream>

```

Граф включаемых заголовочных файлов для VectorCalculator.cpp:



4.42 VectorCalculator.cpp

[См. документацию.](#)

```

00001 #include "VectorCalculator.h"
00002 #include <limits>
00003 #include <climits>
00004 #include <iostream>

```



```

00005
00012 uint64_t VectorCalculator::computeProduct(const std::vector<uint64_t>& vector) {
00013     if (vector.empty()) {
00014         return 0;
00015     }
00016
00017     uint64_t product = 1;
00018
00019     for (uint64_t value : vector) {
00020         if (value == 0) {
00021             return 0;
00022         }
00023
00024         // Check for overflow for uint64_t
00025         if (product > UINT64_MAX / value) {
00026             return UINT64_MAX;
00027         }
00028
00029         product *= value;
00030     }
00031
00032     return product;
00033 }

```

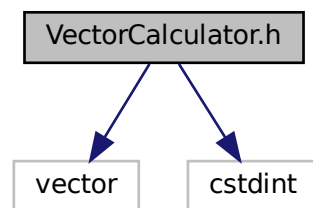
4.43 Файл VectorCalculator.h

```

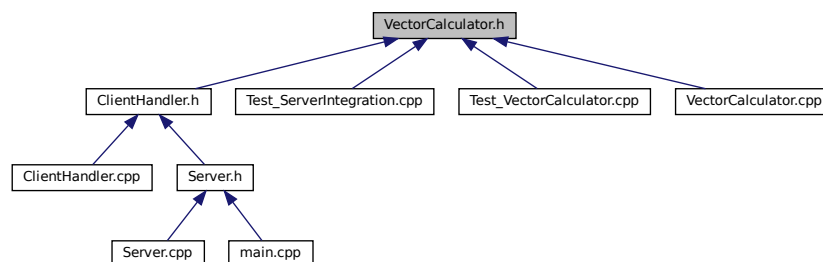
#include <vector>
#include <cstdint>

```

Граф включаемых заголовочных файлов для VectorCalculator.h:



Граф файлов, в которые включается этот файл:



Классы

- class [VectorCalculator](#)

Класс для вычисления произведения элементов вектора

4.44 VectorCalculator.h

[См. документацию.](#)

```
00001 #ifndef VECTORCALCULATOR_H
00002 #define VECTORCALCULATOR_H
00003
00004 #include <vector>
00005 #include <stdint>
00006
00014 class VectorCalculator {
00015 public:
00022     static uint64_t computeProduct(const std::vector<uint64_t>& vector);
00023 };
00024
00025 #endif
```

Предметный указатель

- ~ClientBaseFixture
 - ClientBaseFixture, 11
- ~Logger
 - Logger, 23
- ~LoggerFixture
 - LoggerFixture, 28
- ~OutputRedirect
 - OutputRedirect, 29
- ~Server
 - Server, 32
- authenticate
 - Authenticator, 6
- authenticateClient
 - ClientHandler, 13
- Authenticator, 5
 - authenticate, 6
 - computeSHA1, 6
 - generateSalt, 6
- Authenticator.cpp, 39
- Authenticator.h, 40
- be64toh
 - ClientHandler.cpp, 44
- buffer
 - OutputRedirect, 30
- cleanup
 - Server, 32
- clear
 - OutputRedirect, 29
- clientAddr
 - ClientHandler, 18
- ClientBase, 7
 - ClientBase, 8
 - clients, 10
 - filename, 10
 - getPassword, 8
 - load, 9
 - userExists, 9
- clientBase
 - ClientHandler, 18
 - Server, 33
- ClientBase.cpp, 41
- ClientBase.h, 42
- clientBaseFile
 - Server, 33
 - ServerConfig, 35
- ClientBaseFixture, 10
 - ~ClientBaseFixture, 11
- ClientBaseFixture, 10
 - testFilename, 11
- ClientHandler, 11
 - authenticateClient, 13
 - clientAddr, 18
 - clientBase, 18
 - ClientHandler, 13
 - clientSocket, 18
 - handle, 13
 - logger, 18
 - receiveData, 14
 - receiveString, 14
 - receiveVectorsAndProcess, 15
 - sendData, 16
 - sendMessage, 17
 - sendSalt, 17
- ClientHandler.cpp, 44
 - be64toh, 44
 - htobe64, 44
- ClientHandler.h, 48
- clients
 - ClientBase, 10
- clientSocket
 - ClientHandler, 18
- CommandLineParser, 19
 - CommandLineParser, 20
 - config, 21
 - getConfig, 20
 - getHelp, 20
 - helpText, 21
 - parse, 21
- CommandLineParser.cpp, 49
- CommandLineParser.h, 51
- computeProduct
 - VectorCalculator, 37
- computeSHA1
 - Authenticator, 6
- config
 - CommandLineParser, 21
- ERROR
 - Logger.h, 55
- error
 - Logger, 24
- filename
 - ClientBase, 10
- generateSalt
 - Authenticator, 6

- getConfig
 - CommandLineParser, 20
- getCurrentTime
 - Logger, 24
- getHelp
 - CommandLineParser, 20
- getOutput
 - OutputRedirect, 29
- getPassword
 - ClientBase, 8
- handle
 - ClientHandler, 13
- helpText
 - CommandLineParser, 21
- htobe64
 - ClientHandler.cpp, 44
- INFO
 - Logger.h, 55
- info
 - Logger, 24
- initialize
 - Server, 32
- levelToString
 - Logger, 25
- load
 - ClientBase, 9
- log
 - Logger, 25
- logFile
 - Logger, 27
 - Server, 34
 - ServerConfig, 35
- Logger, 22
 - ~Logger, 23
 - error, 24
 - getCurrentTime, 24
 - info, 24
 - levelToString, 25
 - log, 25
 - logFile, 27
 - Logger, 23
 - logMutex, 27
 - warning, 25
- logger
 - ClientHandler, 18
 - Server, 34
- Logger.cpp, 52
- Logger.h, 54
 - ERROR, 55
 - INFO, 55
 - LogLevel, 55
 - WARNING, 55
- LoggerFixture, 27
 - ~LoggerFixture, 28
 - LoggerFixture, 28
 - readLogFile, 28
- testLogFile, 28
- LogLevel
 - Logger.h, 55
- logMutex
 - Logger, 27
- main
 - main.cpp, 56
 - test_main.cpp, 69
- main.cpp, 56
 - main, 56
 - server, 57
 - signalHandler, 57
- old_cerr
 - OutputRedirect, 30
- old_cout
 - OutputRedirect, 30
- OutputRedirect, 29
 - ~OutputRedirect, 29
 - buffer, 30
 - clear, 29
 - getOutput, 29
 - old_cerr, 30
 - old_cout, 30
 - OutputRedirect, 29
- parse
 - CommandLineParser, 21
- port
 - Server, 34
 - ServerConfig, 36
- readLogFile
 - LoggerFixture, 28
- receiveData
 - ClientHandler, 14
- receiveString
 - ClientHandler, 14
- receiveVectorsAndProcess
 - ClientHandler, 15
- run
 - Server, 33
- running
 - Server, 34
- sendData
 - ClientHandler, 16
- sendMessage
 - ClientHandler, 17
- sendSalt
 - ClientHandler, 17
- Server, 30
 - ~Server, 32
 - cleanup, 32
 - clientBase, 33
 - clientBaseFile, 33
 - initialize, 32
 - logFile, 34

- logger, [34](#)
- port, [34](#)
- run, [33](#)
- running, [34](#)
- Server, [32](#)
- serverSocket, [34](#)
- stop, [33](#)
- server
 - main.cpp, [57](#)
- Server.cpp, [58](#)
- Server.h, [60](#)
- ServerConfig, [35](#)
 - clientBaseFile, [35](#)
 - logFile, [35](#)
 - port, [36](#)
- serverSocket
 - Server, [34](#)
- signalHandler
 - main.cpp, [57](#)
- stop
 - Server, [33](#)
- SUITE
 - Test_Authenticator.cpp, [62](#)
 - Test_ClientBase.cpp, [63](#)
 - Test_CommandLineParser.cpp, [65](#)
 - Test_Logger.cpp, [67](#)
 - Test_ServerIntegration.cpp, [70](#)
 - Test_VectorCalculator.cpp, [71](#)
- Test_Authenticator.cpp, [61](#)
 - SUITE, [62](#)
- Test_ClientBase.cpp, [62](#)
 - SUITE, [63](#)
- Test_CommandLineParser.cpp, [64](#)
 - SUITE, [65](#)
- Test_Logger.cpp, [67](#)
 - SUITE, [67](#)
- test_main.cpp, [69](#)
 - main, [69](#)
- Test_ServerIntegration.cpp, [70](#)
 - SUITE, [70](#)
- Test_VectorCalculator.cpp, [71](#)
 - SUITE, [71](#)
- testFilename
 - ClientBaseFixture, [11](#)
- testLogFile
 - LoggerFixture, [28](#)
- userExists
 - ClientBase, [9](#)
- VectorCalculator, [36](#)
 - computeProduct, [37](#)
- VectorCalculator.cpp, [72](#)
- VectorCalculator.h, [73](#)
- WARNING
 - Logger.h, [55](#)
- warning
 - Logger, [25](#)